

Interview questions and their perspective answers

What is the class in java?

A class in Java is a blueprint used to create objects. It defines variables and methods that represent the properties and behaviour of those objects, and it is used to implement object-oriented concepts like encapsulation, inheritance, and polymorphism.

What is method?

A method in Java is a function defined inside a class that contains a set of instructions to perform a specific task. It represents the behaviour of an object and is executed when it is called.

What is an object and how many ways to create objects?

An object in Java is an instance of a class that represents a real-world entity and contains data (variables) and behaviour (methods).

An object is created using the new keyword along with the class name.

Example:

```
Car c1 = new Car();
```

In Java, there are mainly 5 ways to create an object:

1. Using the new keyword
2. Using `Class.forName().newInstance()`
3. Using `clone()` method
4. Using object deserialization
5. Using `Constructor.newInstance()` (Reflection)

How to initialize object ?

Answer

An object in Java is initialized in three main ways:

1. Using a reference variable after object creation
2. Using a method
3. Using a constructor

Example:

```
Car c1 = new Car(); // object creation
```

```
c1.color = "Red"; // initialization using reference
```

 Importance

 Medium

what is constructor. and how many access modifier can we use in it.

 Answer

A constructor in Java is a special method used to initialize an object. It has the same name as the class and is called automatically when an object is created.

We can use all access modifiers with a constructor: public, private, protected, and default (no modifier). Constructor does not have any return type not even void.

 Importance

 High

can we use static, final, synchronized in constructor.

 Answer

No, we cannot use static, final, or synchronized with a constructor in Java because a constructor is used only for object initialization and is called during object creation, not like a regular method.

 Importance

 High

The default value of String is null and int is 0.

Use of constructor - To initialize the object, not to create an object.

how many types of constructors and who is responsible to create them?

 Answer

In Java, there are mainly two types of constructors: default constructor and parameterized constructor. The default constructor is created by the compiler if no constructor is defined, and the parameterized constructor is created by the programmer.

 Importance

 High

and what about no args const

 Answer

A no-argument constructor is a constructor that does not take any parameters. It can be created either by the programmer explicitly or automatically by the compiler if no constructor is defined in the class.

 Importance

 High

what is inheritance

 Answer

Inheritance in Java is a mechanism by which one class acquires the properties and behaviors (variables and methods) of another class using the extends keyword, promoting code reusability and hierarchical relationships.

 Importance

 High

what type of relationship is of inheritance

 Answer

Inheritance represents an **IS-A relationship** in Java, where one class is a type of another class.

 Importance

 High

Type of inheritance in java ?

 Answer

In Java, there are five types of inheritance:

1. Single inheritance
2. Multilevel inheritance
3. Hierarchical inheritance
4. Multiple inheritance (not supported with classes, supported using interfaces)
5. Hybrid inheritance (achieved using interfaces)

 Importance

● High

which class is the parent class of all the class in java.

🎯 Answer

The parent class of all classes in Java is the Object class from the java.lang package.

🎯 Importance

● High

why is multiple and hybrid inheritance not supported in java and why supported by interface only?

🎯 Answer

Multiple and hybrid inheritance are not supported in Java using classes to avoid ambiguity problems like the diamond problem, where it becomes unclear which method to inherit when multiple parent classes have the same method.

Interfaces support multiple inheritance because they do not contain method implementation (earlier) and even with default methods, Java provides clear rules to resolve conflicts, so ambiguity is avoided.

🎯 Importance

● High

advantage and disadvantage of inheritance

🎯 Answer

Advantages of Inheritance:

- Promotes code reusability
- Reduces code duplication
- Improves maintainability
- Supports hierarchical classification

Disadvantages of Inheritance:

- Creates tight coupling between classes
- Changes in parent class can affect child classes
- Can increase complexity in large hierarchies
- May lead to misuse if not designed properly

Importance

 High

which does not take part in inheritance

Answer

Constructors do not take part in inheritance in Java because they are used only to initialize objects of their own class and are not inherited by child classes.

Importance

 High

and private members

Answer

Private members do not take part in inheritance in Java because they are accessible only within their own class and cannot be accessed directly by child classes.

Importance

 High

so how can we access it

Answer

Private members can be accessed indirectly in a child class using public or protected getter and setter methods defined in the parent class.

Importance

 High

relationship between classes

Answer

The main relationships between classes in Java are:

1. **IS-A relationship** (Inheritance)
2. **HAS-A relationship** (Composition/Aggregation)

Importance

 High

tell me more about relationship in brief

Answer

In Java, there are mainly two types of relationships between classes:

IS-A relationship and **HAS-A relationship**.

IS-A represents inheritance, where one class extends another.

HAS-A represents composition or aggregation, where one class contains an object of another class.

These relationships help in code reusability and better design.

Importance

 High

Alright — now let's step up from interview mode to **concept clarity mode (but still crisp and structured)**.

Relationships Between Classes in Java (Complete Brief)

In Java, relationships define how classes interact with each other. There are **3 main types**:

1. IS-A Relationship (Inheritance)

- Represents **inheritance**
- One class is a type of another class
- Achieved using extends

 Example:

```
class Animal {}
```

```
class Dog extends Animal {}
```

✓ Dog **IS-A** Animal

Use when: You want to reuse code and create hierarchy

2. HAS-A Relationship (Composition & Aggregation)

Represents **containment** → one class has another class as a member

Composition (Strong relationship)

- Child **cannot exist without parent**
- Strong dependency

 Example:

```
class Engine {}
```

```
class Car {  
  
    Engine engine = new Engine();  
  
}
```

✓ Car **HAS-A** Engine

! If Car is destroyed → Engine is also gone

◆ Aggregation (Weak relationship)

- Objects can exist **independently**
- Loose coupling

👉 Example:

```
class Student {}  
  
class College {  
  
    Student student;  
  
}
```

✓ College **HAS-A** Student

✓ Student can exist without College

3. **USES-A Relationship (Dependency)**

- Temporary relationship
- One class uses another class **for a method or operation**

👉 Example:

```
class A {  
  
    void display(B b) {  
  
        // using B object  
  
    }  
  
}
```

✓ A **USES-A** B

Use when: You only need another class temporarily

⚡ **Quick Summary (Important)**

Relationship	Meaning	Example
IS-A	Inheritance	Dog → Animal
HAS-A	Contains object	Car → Engine
USES-A	Temporary use	Method parameter

💡 **Real Insight (Very Important)**

- **IS-A → Tight coupling (use carefully)**
- **HAS-A → Preferred in real projects (flexible design)**
- **USES-A → Lightweight interaction**

👉 In real-world Spring Boot projects,
HAS-A + Dependency Injection = Best design

See more on my java notes page number 14

what is polymorphism

🎯 **Answer**

Polymorphism in Java is the ability of an object to take multiple forms, meaning the same method can behave differently based on the context, such as method overloading and method overriding.

🎯 **Importance**

● High

how many types of polymorphism and handled by whom

🎯 **Answer**

There are two types of polymorphism in Java:

1. Compile-time polymorphism (method overloading) – handled by the compiler
2. Runtime polymorphism (method overriding) – handled by the JVM

🎯 **Importance**

● High

can we achieve method overloading by changing the return type of method only

🎯 **Answer**

No, method overloading cannot be achieved by changing only the return type because the method signature does not include the return type; it must differ in the number or type of parameters.

Importance

● High

can we overload main method

Answer

“Yes, we can overload the main method in Java by changing its parameter list. However, the JVM only recognizes the standard main method signature `public static void main (String [] args)` as the entry point, so overloaded versions are not executed automatically.”

Importance

● High

method overloading and type promotion and different cases of type promotion and automatic promotion

Answer

Method overloading in Java means defining multiple methods with the same name but different parameter lists.

During overloading, if an exact match is not found, Java applies **type promotion** to find the best match.

Type promotion means automatically converting a smaller data type into a larger data type.

The promotion order is:

byte → short → int → long → float → double

Different cases of type promotion:

- Exact match is preferred first
- If no exact match, the next higher data type is used
- Automatic promotion happens only in one direction (smaller to larger)
- char is promoted to int
- boolean is not promoted

Importance

● High

while resolving overloading methods, compiler will always give precedence for the child type argument then compared to with parent type argument. rewrite with clean and clear

Answer

While resolving overloaded methods, the compiler always gives priority to the child type argument over the parent type argument because it is more specific.

Importance

 High

what does this mean

Answer

It means when multiple overloaded methods are available, the compiler will choose the method whose parameter type is more specific (child class) rather than a general one (parent class).

```
<> Java

class Animal {}
class Dog extends Animal {}

void show(Animal a) {
    System.out.println("Animal");
}

void show(Dog d) {
    System.out.println("Dog");
}

show(new Dog()); // Output: Dog
```

✓ Here, both methods can accept Dog

✓ But compiler chooses Dog version because it is more specific

Importance

 High

tell me about varargs

Answer

Varargs in Java allow a method to accept a any number of arguments of the same type. It is represented using three dots (...) and is treated as an array inside the method.

Example:

```
void display(int... numbers) {
    for (int n : numbers) {
        System.out.println(n);
    }
}
```

```
}  
}
```

🎯 Importance

● High

different between overloading and overriding

🎯 Answer

Method Overloading means defining multiple methods with the same name but different parameters within the same class, and it is resolved at compile time.

Method Overriding means redefining a method of the parent class in the child class with the same signature, and it is resolved at runtime.

🎯 Importance

● High

do overriding method must have same return type or subtype

👉 Interview-ready answer:

"An overriding method must have the same return type or a subtype of the original method's return type. This is known as covariant return type, where the subclass can return a more specific type while maintaining compatibility with the parent method."

👉 Example (Covariant Return Type):

```
</> Java  
  
class Animal {}  
  
class Dog extends Animal {}  
  
class Parent {  
    Animal getAnimal() {  
        return new Animal();  
    }  
}  
  
class Child extends Parent {  
    @Override  
    Dog getAnimal() { // ✅ subtype return type (Dog is a subtype of Animal)  
        return new Dog();  
    }  
}
```

👉 Interview one-liner:

"In method overriding, the return type can be the same or a subtype, as shown where the parent returns Animal and the child returns Dog."

Importance

● High

Overriding and access modifiers.

Answer

In method overriding, the access modifier of the overriding method cannot be more restrictive than the parent method; it can be the same or more accessible.

Example:

- protected → can be overridden as protected or public
- public → must remain public

Importance

● High

overriding and exception handling

Answer

If the super class overridden method does not throw an exception, subclass overriding method can only throw the unchecked exception, throwing checked exception will lead to compile-time-error.

If the super class overridden method does throw an exception, subclass overriding method can only throw the same exception. Throwing bigger parent exception in exception hierarchy will lead to compile-time-error. Also, there is no issue if sub class overridden method is not throwing any exception.

Importance

● High

overriding and abstract method

Answer

An abstract method must be overridden in a subclass, and the overriding method must provide its implementation. If it does not override the abstract method, then the subclass must be declared abstract.

Abstract methods in an interface or abstract class are meant to be overridden in derived concrete classes otherwise compile-time-error will be thrown.

Importance

 High

invoking overridden method from sub class

Answer

An overridden method from the parent class can be invoked in the subclass using the super keyword.

Example:

```
class Parent {  
void show() {  
System.out.println("Parent method");  
}  
}
```

```
class Child extends Parent {  
void show() {  
super.show(); // calling parent method  
System.out.println("Child method");  
}  
}
```

Importance

 High

which method cannot be override

Answer

In Java, the following methods cannot be overridden:

- final methods
- static methods
- private methods

Importance

 High

overriding and synchronized strictfp method

Answer

In method overriding, synchronized and strictfp modifiers are not mandatory to be the same as the parent method; the child method can choose to include or omit them independently.

Importance

 Medium

what is abstraction more details

Answer

Abstraction in Java is the process of hiding implementation details and showing only essential features to the user. It helps focus on what an object does rather than how it does it. Abstraction is achieved using abstract classes and interfaces.

Importance

 High

what is getter and setter

Answer

Getter and setter are methods used to access and update the value of private variables in a class. Getter is used to read the value, and setter is used to modify the value, helping to achieve encapsulation.

Importance

 High

give one example

Answer

```
class Person {
    private String name; // private variable

    // Getter
    public String getName() {
        return name;
    }

    // Setter
    public void setName(String name) {
        this.name = name;
    }
}

Usage:

// Java
Person p = new Person();
p.setName("Yasif"); // set value
System.out.println(p.getName()); // get value
```

Importance

 High

multiple and hybrid are supported by interfaces. so, what if we have same default method in multiple parent classes

Answer

If multiple interfaces have the same default method, the implementing class must override that method to resolve the ambiguity.

```
interface A {
    default void show() {
        System.out.println("A");
    }
}

interface B {
    default void show() {
        System.out.println("B");
    }
}

class Test implements A, B {
    public void show() { // must override
        System.out.println("Resolved");
    }
}
```

Importance

 High

so, we must override. and without override we cannot call it because it will confuse the compiler

Answer

Yes, we must override the method because otherwise the compiler cannot decide which default method to use, leading to ambiguity (diamond problem). Without overriding, it will result in a compile-time error.

Importance

● High

what is diamond problem

🎯 Answer

The diamond problem is a situation in multiple inheritance where a class inherits from two parent classes that have the same method, causing ambiguity about which method should be used.

```
</> Java

interface A {
    default void show() {
        System.out.println("A");
    }
}

interface B extends A {}
interface C extends A {}

class D implements B, C {
    // ambiguity: which show() to use?
}
```

✓ To resolve this in Java, the class must override the method.

🎯 Importance

● High

can we print hello without main method

👉 Interview-ready answer:

“No, in modern Java we cannot run a program without the main method because the JVM requires it as the entry point. However, we can still execute code like printing ‘Hello’ using static blocks or other mechanisms, but the class must contain a main method for the JVM to start execution. So practically, the main method is mandatory.”

can we override private method

👉 Interview-ready answer:

“No, we cannot override a private method in Java because private methods are not inherited by subclasses. Since overriding requires inheritance, the subclass cannot access the private method to override it. If a subclass defines a method with the same name, it is treated as a completely new method, not an overridden one.”

tell me about interface

👉 Interview answer:

“An interface in Java is a blueprint of a class that contains abstract methods and constants variables. It is used to achieve abstraction and multiple inheritance, where a class implements the interface and provides the method definitions. With modern Java, interfaces can also have default, static, and private methods.”

what is encapsulation

👉 Interview-ready answer:

“Encapsulation is the process of wrapping data and methods into a single unit, basically a class, and restricting direct access to the data by making variables private and providing controlled access through getters and setters. It helps in data hiding, improves security, and maintains better control over the code.”

what is data hiding

👉 Interview-ready answer:

“Data hiding is the concept of restricting direct access to the data by making variables private and providing controlled access through getters and setters. It helps in data hiding, improves security, and maintains better control over the code.”

Does a Class Occupy Memory in Java?

Short Answer

Yes — a class occupies memory, but **not in the Heap**. It is stored in the **Method Area (Metaspace)** of the JVM when loaded. Only **objects (instances)** of a class occupy Heap memory.

Full Flow: From Source Code to Memory

YourCode.java

↓ (javac compiler)

YourCode.class ← Bytecode (platform-independent)

↓ (ClassLoader)

JVM Memory ← Class is now loaded into Metaspace

↓ (new keyword)

Heap Memory ← Object instance is created

JVM Memory Areas — Where Everything Lives

Memory Area	What It Stores
Metaspace (Method Area)	Class metadata, method bytecode, static variables, constant pool
Heap Memory	Object instances (created via new)
Stack Memory	Method calls, local variables, reference variables
PC Register	Address of current executing instruction (per thread)
Native Method Stack	Native (non-Java) method calls

The Class Loading Process

When a .class file is loaded by the JVM, it goes through **3 phases**:

1. Loading

- The **ClassLoader** reads the .class file from disk (or network/jar)
- Creates a Class object representing the class in memory
- Stores class metadata in **Metaspace**

2. Linking

Three sub-steps:

- **Verification** — checks bytecode is valid and safe
- **Preparation** — allocates memory for static variables and sets default values
- **Resolution** — replaces symbolic references with direct memory references

3. Initialization

- Executes **static blocks** (static { })
- Assigns actual values to **static variables**

Java's 3 Built-in ClassLoaders

ClassLoader	Responsibility
Bootstrap ClassLoader	Loads core Java classes (java.lang.*, java.util.*, etc.)

Extension ClassLoader Loads classes from jre/lib/ext

Application ClassLoader Loads your own .class files and classpath libraries

They follow a **Parent Delegation Model** — before loading a class, a ClassLoader first delegates to its parent. This prevents duplicate loading and protects core Java classes.

What Exactly Is Stored in Metaspace for a Class?

When a class is loaded, the following are stored in **Metaspace**:

- Class name, package, parent class, implemented interfaces
- Method bytecode (all methods of the class)
- Static variables and their values
- Constant pool (string literals, class references, etc.)
- Constructor definitions
- Field definitions (names, types, modifiers)

What Goes to Heap Memory?

Only when you do:

```
Car c = new Car();
```

- The **object** (new Car()) is created in **Heap**
- The **reference variable** c is stored in **Stack**
- The **class metadata** of Car is already in **Metaspace**

Code Example

```
class Car {  
  
    static int count = 0; // → Metaspace (static variable)  
  
    String color; // → Heap (per object, when instantiated)  
  
    static {  
  
        System.out.println("Class loaded!"); // runs during Initialization phase  
    }  
  
    Car(String color) {
```

```

this.color = color;

count++;

}

}

public class Main {

public static void main(String[] args) {

// At this point, Car.class is already loaded into Metaspace

// No heap memory used yet for Car

Car c1 = new Car("Red"); // Object 1 → Heap

Car c2 = new Car("Blue"); // Object 2 → Heap

// c1, c2 are reference variables → Stack

// Car.count (static) → Metaspace

}

}

```

Metaspace vs PermGen (Important for Java 8+)

	PermGen (Before Java 8)	Metaspace (Java 8+)
Location	JVM Heap (fixed size)	Native OS Memory
Size	Fixed, caused OutOfMemoryError easily	Grows dynamically
GC	Part of Full GC	Managed separately
Default max size	Limited	Unlimited (can be capped with -XX:MaxMetaspaceSize)

Java 8 replaced **PermGen** with **Metaspace** to solve frequent java.lang.OutOfMemoryError: PermGen space errors.

Summary

Question	Answer
----------	--------

Does a class occupy memory?	Yes
Where?	Metaspace (Method Area)
What file is loaded?	.class file (bytecode)
Who loads it?	ClassLoader
When is it loaded?	First time the class is referenced at runtime
Do objects occupy memory?	Yes, in Heap Memory
Is class loaded once or many times?	Once per ClassLoader (reused after that)

One-Line Interview Answer

"A class itself is loaded into Metaspace by the ClassLoader when first referenced. Its static data and metadata live there. Only object instances created using new, occupy Heap memory."

tell me about this keyword

Answer

"The this keyword in Java is a reference variable that refers to the current object of the class. It is used to access instance variables, invoke current class methods or constructors, and resolve ambiguity between local variables and instance variables."

Importance

 High

super keyword

Answer

"The super keyword in Java is a reference variable used to refer to the immediate parent class object. It is mainly used to access parent class variables, invoke parent class methods, and call the parent class constructor."

Importance

 High

difference between super and super ()

Answer

super is a reference variable used to access parent class variables and methods, whereas super () is used to call the parent class constructor.

Importance

 High

this and this ()

Answer

this is a reference variable used to refer to the current class object and access its variables and methods, whereas this () is used to call another constructor of the same class.

Importance

 High

final keyword

Answer

“The final keyword in Java is used to restrict modification. A final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be extended. It helps in creating immutable and secure code.”

Importance

 High

Interview Closing Answer

If asked:

 *“What is static in Java?”*

You say:

“Static in Java is used to create class-level members that are shared across all instances. It helps in memory optimization and allows access without object creation. Common uses include static variables, methods, blocks, and utility classes. However, in modern applications like Spring Boot, overuse of static is avoided in favour of dependency injection.”

why static can't access instance members

👉 Interview-ready answer:

“Static members belong to the class, not to any specific object, whereas instance members belong to objects. static context is created at class loading time and does not depend on any object; it doesn’t have a reference to instance data. That’s why static methods cannot directly access instance members unless an object is explicitly created.”

what will happen when we use static with variable and without static in variable

👉 Interview-ready answer:

“When a variable is declared as static, only one copy is created and shared among all objects of the class, so any change made by one object is reflected across all others. Without static, each object has its own separate copy of the variable, so changes in one object do not affect others. Static variables are used for common shared data, while non-static variables are used for object-specific data.”

static variable is used for memory management. Static keyword can be use in variables in class level, methods, blocks, and inner class (nested class). And not used in local variables and outer class.

tell me about static method

👉 Interview-ready answer:

“A static method in Java is a method that belongs to the class rather than an instance, so it can be called without creating an object. It can access only static members directly and is commonly used for utility or helper functions. Static methods improve memory efficiency and are widely used in scenarios where behaviour is not dependent on object state.”

static block

👉 Interview-ready answer:

“A static block in Java is a block of code that is executed once when the class is loaded into memory. It is mainly used for initializing static variables or performing one-time setup tasks. Static blocks run before the main method and before any object of the class is created.”

? Can static methods access non-static members?

👉 Answer:

“No, static methods cannot directly access non-static members because they belong to the class and do not have a reference to any object. To access non-static members, an object must be created.”

? Can we override static methods?

👉 Answer:

“No, static methods cannot be overridden because they belong to the class. If a subclass defines the same static method, it results in method hiding, not overriding.”

? Can we overload static methods?

👉 Answer:

“Yes, static methods can be overloaded by changing their parameter list, as overloading is resolved at compile time and does not depend on object instances.”

? Can static methods use this keyword?

👉 Answer:

“No, static methods cannot use the ‘this’ keyword because ‘this’ refers to the current object, and static methods do not operate on any object.”

? Can constructors be static?

👉 Answer:

“No, constructors cannot be static because constructors are used to initialize objects, and static belongs to the class level, not to instances.”

? Can we declare static variables inside methods?

👉 Answer:

“No, Java does not allow static variables inside methods because static variables belong to the class level, not to method scope.”

? When should we use static?

👉 Answer:

“We should use static when data or behaviour is common across all instances, such as constants, utility methods, or shared resources, to improve memory efficiency and maintain consistency.”

? Difference between primitive and non-primitive data types

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer:

“Primitive data types are basic data types that store simple values directly, such as int, float, and char, and they are stored in stack memory with fixed size. Non-primitive data types, also known as reference types, store references to objects rather than actual data, such as arrays, classes, and strings, and they are stored in heap memory. Primitive types are faster and memory efficient, while non-primitive types are more flexible and support methods and object-oriented features.”

? What are Wrapper Classes in Java?

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer:

“Wrapper classes in Java are used to convert primitive data types into objects. Each primitive type has a corresponding wrapper class, such as int to Integer and char to Character. They are useful when working with collections, generics, and APIs that require objects instead of primitive values, and they also provide utility methods for data conversion and manipulation.”

? What is Autoboxing and Unboxing in Java?

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer:

“Autoboxing is the automatic conversion of a primitive data type into its corresponding wrapper class object, such as converting int to Integer. Unboxing is the reverse process, where a wrapper object is automatically converted back to its primitive type. This feature simplifies code when working with collections and APIs that require objects.”

? Autoboxing & Unboxing Example

 Importance: ★★★★★ (Very High)

👉 Example:

```
</> Java

int a = 10;

// Autoboxing (primitive → object)
Integer obj = a;

// Unboxing (object → primitive)
int b = obj;
```

👉 Interview one-liner:

“In this example, assigning int to Integer is autoboxing, and assigning Integer back to int is unboxing.”

? What is a Variable in Java?

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer:

“A variable in Java is a named memory location used to store data that can be modified during program execution. It has a specific data type which defines the type of value it can hold, and it helps in managing and manipulating data within a program.”

? Types of Variables in Java (with example)

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer:

“In Java, variables are mainly of three types: local variables, instance variables, and static variables. Local variables are declared inside methods and exist only during method execution, instance variables belong to objects and are unique for each instance, and static variables belong to the class and are shared among all objects.”

👉 **Example:**

```
class Demo {  
  
    static int staticVar = 100;    // static variable (class level)  
  
    int instanceVar = 50;          // instance variable (object level)  
  
    void show() {  
        int localVar = 10;        // local variable (method level)  
  
        System.out.println(localVar);  
        System.out.println(instanceVar);  
        System.out.println(staticVar);  
    }  
  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.show();  
    }  
}
```

👉 **Interview one-liner:**

“Local variables are method-level, instance variables are object-level, and static variables are class-level shared across all objects.”

? What are Operators in Java?

📊 Importance: ★★★★★ (Very High)

👉 **Interview-ready answer:**

“Operators in Java are special symbols used to perform operations on variables and values. They are used for arithmetic calculations, comparisons, logical decisions, and other operations within a program.”

🔥 Types of Operators (with example)

```
</> Java

int a = 10, b = 5;

// Arithmetic
int sum = a + b;          // +, -, *, /, %

// Relational
boolean res = a > b;      // >, <, ==, !=

// Logical
boolean check = (a > 5 && b < 10); // &&, ||, !

// Assignment
a += b;                  // =, +=, -=

// Unary
a++;                     // ++, --

// Ternary
int max = (a > b) ? a : b;
```

👉 Interview one-liner:

“Operators are categorized into arithmetic, relational, logical, assignment, unary, and ternary operators, each used for specific operations in Java programs.”

? What are Literals in Java?

📖 Importance: ★★★★★ (High)

👉 Interview-ready answer:

“Literals in Java are fixed values assigned to variables directly in the code. They represent constant values such as numbers, characters, strings, or boolean values, and do not change during program execution.”

👉 **Example:**

```
int a = 10;    // integer literal
```

```
double b = 5.5; // floating literal
```

```
char c = 'A';  // character literal
```

```
String s = "Hello"; // string literal
```

```
boolean flag = true; // boolean literal
```

👉 **Interview one-liner:**

“Literals are constant values directly written in the code, like 10, 'A', or 'Hello'.”

❓ **What are Keywords and Identifiers in Java?**

📖 **Importance:** ★★★★★ (Very High)

👉 **Interview-ready answer:**

“Keywords are reserved words in Java that have predefined meanings and cannot be used as variable or class names, such as `int`, `class`, and `static`. Identifiers are user-defined names given to variables, methods, classes, or objects, and they are used to identify these elements in a program.”

👉 **Example:**

```
int number = 10;
```

- `int` → keyword
 - `number` → identifier
-

👉 **Interview one-liner:**

“Keywords are predefined reserved words, while identifiers are user-defined names used to represent variables, methods, and classes.”

? What are Naming Conventions in Java?

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer:

“Naming conventions in Java are a set of standardized rules used for naming variables, methods, classes, and other identifiers to improve code readability and maintainability. For example, class names follow PascalCase, method and variable names follow camelCase, and constants are written in uppercase with underscores. Following naming conventions makes the code more consistent, understandable, and easier to collaborate on in real-world projects.”

example

? Naming Conventions Example

 Importance: ★★★★★ (Very High)

👉 Example:

```
class StudentDetails {           // Class → PascalCase

    private int studentAge;       // Variable → camelCase

    public void calculateMarks() { // Method → camelCase
        int totalMarks = 100;    // Local variable → camelCase
    }

    public static final double PI_VALUE = 3.14; // Constant → UPPER_CASE
}
```

👉 Interview-ready answer:

“In this example, the class name follows PascalCase, variables and methods follow camelCase, and constants are written in uppercase with underscores. These conventions improve readability, maintain consistency, and make the code easier to understand and maintain in real-world applications.”

? Java Introduction, History, Features & Uses

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer:

“Java is a high-level, object-oriented, and platform-independent programming language designed to be simple, secure, and portable. It follows the principle of ‘write once, run anywhere,’ meaning compiled Java code can run on any system that has a Java Virtual Machine (JVM).

Java was developed by James Gosling at Sun Microsystems in 1995. It was initially called Oak and later renamed Java. After Sun Microsystems, Java is now maintained by Oracle Corporation.

Java provides several important features such as platform independence, object-oriented programming, robustness, security, multithreading, and high performance with Just-In-Time compilation. It also has automatic memory management through garbage collection, which makes development easier and reduces errors.

Java is widely used in building enterprise applications, web applications using frameworks like Spring Boot, mobile applications through Android, and large-scale distributed systems. It is also used in cloud-based applications, banking systems, and backend development due to its scalability and reliability.”

? Difference between Java, C++, and Python

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer:

“Java, C++, and Python are popular programming languages but differ in design and usage. Java is a platform-independent, object-oriented language that runs on the JVM and is widely used for enterprise and backend applications. C++ is a high-performance, compiled language that supports both procedural and object-oriented programming and is commonly used in system-level programming and game development. Python is an interpreted, high-level language known for its simplicity and readability, widely used in data science, scripting, and rapid development. Compared to C++, Java provides better memory management through garbage collection, and compared to Python, Java offers better performance and strong type safety, making it suitable for large-scale applications.”

? Java Architecture (JVM, JRE, JDK)

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer:

“Java architecture consists of three main components: JDK, JRE, and JVM. The JDK, or Java Development Kit, provides tools required to develop Java applications, such as the compiler and

debugger. The JRE, or Java Runtime Environment, provides the libraries and environment needed to run Java programs. The JVM, or Java Virtual Machine, is responsible for executing Java bytecode and converting it into machine code for the underlying system. This architecture enables Java's platform independence, as the same bytecode can run on any system with a compatible JVM."

👉 If interviewer asks follow-up, you can add:

"The JVM also handles memory management, garbage collection, and security, making Java robust and reliable for enterprise applications."

? JDK (Java Development Kit)

 Importance: ★★★★★ (Very High)

👉 Answer:

"JDK is a complete development kit used to build Java applications. It includes tools like the compiler (javac), debugger, and other utilities required for development. It also contains JRE, so it provides both development and execution environment."

? JRE (Java Runtime Environment)

 Importance: ★★★★★ (Very High)

👉 Answer:

"JRE provides the environment required to run Java programs. It includes JVM and core libraries but does not contain development tools like the compiler. It is mainly used for executing Java applications."

? JVM (Java Virtual Machine)

 Importance: ★★★★★ (Very High)

👉 Answer:

"JVM is an abstract machine that executes Java bytecode and converts it into machine code. It is responsible for platform independence, memory management, garbage collection, and security during program execution."

🧠 Java Architecture (Simple Flow)

👉 Interview-ready explanation:

"In Java architecture, the developer writes code which is compiled by the JDK into bytecode. This bytecode is then executed by the JVM inside the JRE. The JVM converts the bytecode into machine-specific instructions, allowing the program to run on any platform. This is how Java achieves platform independence."

👉 Flow to say in interview:

.java file → (JDK compiler) → .class (bytecode) → (JVM in JRE) → Machine Code → Output

? Explain Java Hello World Program

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer:

"A Java Hello World program is the basic program used to demonstrate the structure of a Java application. It includes a class definition and a main method, which acts as the entry point of the program. The main method is declared as public so it can be accessed by the JVM, static so it can be called without creating an object, and void because it does not return any value. Inside the main method, we use System.out.println to print output to the console. System is a predefined class, out is a static object of PrintStream, and println is a method used to display text. This program helps understand how Java code is compiled into bytecode and executed by the JVM."

👉 Example:

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

👉 How you can explain step-by-step in interview (if asked deeper):

- class Hello → defines the class

- `public static void main(String[] args)` → entry point of program
 - `System.out.println()` → prints output
-

? What changes are acceptable for main method?

 Importance: ★★★★★ (Very High)

 Interview-ready answer:

“The main method in Java must follow the signature `public static void main(String[] args)` for the JVM to recognize it as the entry point. However, some minor variations are acceptable, such as changing the parameter name from `args` to any valid identifier, using `String... args` instead of `String[] args`, and changing the order of modifiers like `static public`. These do not affect execution. But the return type must remain `void`, and the method must be `public` and `static`; otherwise, the JVM will not execute the program.”

? Types of Virtual Machine

 Importance: ★★★★★ (High)

 Interview-ready answer:

“Virtual machines are mainly of two types: System Virtual Machine and Process Virtual Machine. A System Virtual Machine provides a complete platform to run an entire operating system, like VMware or VirtualBox. A Process Virtual Machine is designed to run a single program or process, such as the Java Virtual Machine (JVM), which executes Java bytecode. In Java, we mainly deal with the Process Virtual Machine, which enables platform independence and manages execution of Java programs.”

Control statement and its type

? What are Control Statements in Java and their types?

 Importance: ★★★★★ (Very High)

 Interview-ready answer:

“Control statements in Java are used to control the flow of execution of a program based on conditions or loops. They determine which statements will execute and how many times. Control statements are mainly classified into three types: selection statements, iteration statements, and jump statements.”

🔥 Types of Control Statements

◇ 1. Selection (Decision-Making) Statements

👉 Used to make decisions based on conditions

- if
- if-else
- else-if
- switch

```
int a = 10;

if (a > 5) {

    System.out.println("Greater");

}
```

◇ 2. Iteration (Looping) Statements

👉 Used to repeat a block of code

- for
- while
- do-while

```
for (int i = 1; i <= 3; i++) {

    System.out.println(i);

}
```

◇ 3. Jump Statements

👉 Used to transfer control

- break
- continue
- return

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) break;  
}
```

Final Interview Closing Answer

“Control statements in Java are used to manage the flow of execution and are categorized into selection statements for decision-making, iteration statements for looping, and jump statements for transferring control within the program.”

? What are Selection Statements in Java?

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (easy English):

“Selection statements in Java are used to take decisions in a program. They allow the program to choose which block of code to execute based on a condition. If the condition is true, one block runs; if false, another block runs. Common selection statements are if, if-else, else-if, and switch.”

👉 Simple Example:

```
int age = 18;  
  
if (age >= 18) {  
    System.out.println("Eligible to vote");  
} else {  
    System.out.println("Not eligible");  
}
```

👉 In simple words:

“Selection statements help the program make decisions based on conditions.”

? Explain this program (static vs instance behavior)

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready explanation:

“In this program, a is an instance variable and b is a static variable. Instance variables are unique for each object, while static variables are shared among all objects of the class.”

🔍 Step-by-step explanation:

```
class Abc {  
    int a = 10;    // instance variable  
    static int b = 20; // static variable  
}
```

👉 Step 1:

```
Abc obj1 = new Abc();  
System.out.println(obj1.a); // 10  
System.out.println(obj1.b); // 20
```

- a = 10 → each object gets its own copy
 - b = 20 → shared static value
-

👉 Step 2:

```
obj1.a = 1000;  
obj1.b = 2000;
```

- a is changed **only for obj1**
 - b is static → changed **for entire class**
-

👉 Step 3:

```
Abc obj2 = new Abc();
```

```
System.out.println(obj2.a); // 10  
System.out.println(obj2.b); // 2000
```

- obj2.a → gets fresh value **10** (new object)
 - obj2.b → gets updated value **2000** (shared variable)
-

Key Concept

- Instance variable → **separate copy per object**
 - Static variable → **single shared copy**
-

Final Interview Closing Answer

“In this program, the instance variable a is different for each object, so obj2 gets the original value 10. However, the static variable b is shared across all objects, so when it is updated using obj1, the change is reflected in obj2 as well, resulting in 2000.”

compile time exception occurs at compile time and runtime exception occurs at runtime or all the exception occurs at runtime which is right

Compile-time vs Runtime Exceptions — which statement is correct?

 Importance: ★★★★★ (Very High)

Interview-ready answer:

“All exceptions in Java occur at runtime, but compile-time exceptions, also known as checked exceptions, are identified by the compiler during compilation and must be handled using try-catch or declared with throws. Runtime exceptions, also known as unchecked exceptions, occur during program execution and are not checked by the compiler. So, it is correct to say that all exceptions occur at runtime, but some are detected at compile time.”

Clarification (very important concept)

- **Compile-time (Checked) Exceptions**
 - Detected by compiler
 - Example: IOException
- **Runtime (Unchecked) Exceptions**
 - Occur during execution
 - Example: ArithmeticException

👉 **Final Interview Line:**

“Exceptions always occur at runtime, but the compiler forces handling of checked exceptions at compile time, which is why they are called compile-time exceptions.”

❓ **What is Exception and Exception Handling in Java?**

📖 **Importance:** ★★★★★ (Very High)

👉 **Interview-ready answer:**

“An exception in Java is an abnormal condition or error that occurs during program execution and disrupts the normal flow of the program. Exception handling is the mechanism used to handle such runtime errors so that the program can continue executing normally instead of terminating abruptly. It is done using keywords like try, catch, finally, throw, and throws, which allow developers to manage errors in a controlled and structured way.”

❓ **Exception Hierarchy in Java**

📖 **Importance:** ★★★★★ (Very High)

👉 **Interview-ready answer (simple English):**

“In Java, all exceptions are part of a class hierarchy. The root class is Throwable. It has two main subclasses: Error and Exception. Errors are serious problems that are not handled by programs, while Exceptions are conditions that can be handled. Exceptions are further divided into checked exceptions, which are checked at compile time, and unchecked exceptions, which occur at runtime.”

❓ **Difference between Exception and Error**

📖 **Importance:** ★★★★★ (Very High)

👉 **Interview-ready answer (simple English):**

“Exception and Error both come under Throwable, but they are different in nature. Exceptions are problems that can be handled by the program using try-catch blocks, such as IOException or ArithmeticException. Errors are serious issues like OutOfMemoryError or StackOverflowError that occur due to system failure and cannot be handled by the program. Exceptions are recoverable, but Errors are generally not recoverable.”

? Difference between Checked and Unchecked Exceptions

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (simple English):

“Checked exceptions are exceptions that are checked by the compiler at compile time, so they must be handled using try-catch or declared with throws. Unchecked exceptions are not checked by the compiler and occur at runtime, so handling them is optional.”

? Explain what happens when an exception occurs

👉 Clean interview-ready answer:

“When an exception occurs, the JVM creates an exception object that contains the exception type, an error message, and the stack trace showing where the error happened. This object is then passed to the exception handling mechanism to handle the error or terminate the program.”

? Methods to print exception information

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer:

“In Java, we can print exception information using methods like `printStackTrace()`, `getMessage()`, and `toString()`. The `printStackTrace()` method prints the complete details including the stack trace, `getMessage()` returns only the error message, and `toString()` returns the exception type along with the message.”

👉 Example:

```
</> Java

try {
    int a = 10 / 0;
} catch (Exception e) {
    e.printStackTrace(); // full details
    System.out.println(e.getMessage()); // message only
    System.out.println(e.toString()); // type + message
}
```


? What is finally block and its use?

 Importance: ★★★★★ (Very High)

 Interview-ready answer (simple English):

“The finally block in Java is used to execute important code that must run whether an exception occurs or not. It is always executed after the try-catch block, except in rare cases like system shutdown. It is mainly used for resource cleanup, such as closing files, database connections, or streams.”

 Example:

```
try {  
    int a = 10 / 0;  
} catch (Exception e) {  
    System.out.println("Exception occurred");  
} finally {  
    System.out.println("This will always execute");  
}
```

? Can we close resources in try block?

 Importance: ★★★★★ (High)

 Interview-ready answer:

“Yes, we can close resources in the try block, but it is not recommended because if an exception occurs before the close statement, the resource may not get closed. To ensure proper cleanup, it is better to close resources in the finally block or use try-with-resources, which automatically handles resource closing.”

? Rules for finally block

 Importance: ★★★★★ (High)

 Interview-ready answer (simple English):

“The finally block in Java must be used along with a try block, and it can be used with or without a catch block. It always executes after the try or catch block, except in rare cases like when the JVM is

terminated. Only one finally block is allowed per try block, and it is mainly used for cleanup operations like closing resources.”

Key Rules:

- Must be associated with a try block
 - Can be used **with or without catch**
 - Always executes (almost always)
 - Only **one finally block per try**
 - Used for cleanup tasks
-

Final Interview Closing Answer

“The finally block must be associated with a try block, can be used with or without catch, executes almost always after try-catch, and is mainly used for cleanup operations.”

Possibilities that disturb execution of finally block

 Importance: ★★★★★ (High)

Interview-ready answer (simple English):

“The finally block is executed in almost all cases, but there are some situations where it may not execute. These include when the JVM is terminated using System.exit(), when the system crashes or power failure occurs, or when an infinite loop or fatal error prevents normal execution. In such cases, the finally block may not run.”

Key Cases:

- System.exit(0) → JVM stops immediately
 - System crash / power failure
 - Infinite loop before finally
 - Fatal errors (like JVM crash)
-

Final Interview Closing Answer

“Finally block executes in almost all situations, but it may not execute if the JVM is terminated, the system crashes, or the program gets stuck in an infinite loop.”

? Difference between final, finally, and finalize

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (simple English):

“Final, finally, and finalize are different concepts in Java. The final keyword is used to restrict modification, such as making variables constant, methods non-overridable, and classes non-inheritable. The finally block is used in exception handling to execute important code like resource cleanup, and it runs almost always after try-catch. The finalize method is a method of the Object class that is called by the garbage collector before destroying an object, but it is deprecated and not recommended in modern Java.”

🔥 Quick Difference:

Feature	final	finally	finalize
Type	Keyword	Block	Method
Use	Restrict changes	Cleanup code	Before GC
Usage Area	Variables, methods, class	Exception handling	Garbage collection
Execution	At compile time	After try-catch	Before object destruction

? What is used instead of finalize()?

 Importance: ★★★★☆ (High)

👉 Interview-ready answer (clean English):

“The finalize () method is deprecated and not reliable, so it is no longer recommended. Instead, we use try-with-resources or close resources in a finally block to ensure proper cleanup. These approaches are more reliable and widely used in modern Java.”

? What is try-with-resources?

 Importance: ★★★★★ (Very High)

👉 Answer:

“Try-with-resources is used to automatically close resources like files or database connections. It ensures that resources are closed without needing a finally block, making the code cleaner and safer.”

? Can we have multiple catch blocks?

📊 Importance: ★★★★★ (Very High)

👉 Answer:

“Yes, we can have multiple catch blocks to handle different types of exceptions, and they should be ordered from specific to general.”

? What is multi-catch block?

📊 Importance: ★★★★★ (High)

👉 Answer:

“A multi-catch block allows handling multiple exceptions in a single catch block using the pipe (|) operator.”

? What happens if exception is not handled?

📊 Importance: ★★★★★ (Very High)

👉 Answer:

“If an exception is not handled, it is propagated to the JVM, and the program terminates abnormally.”

? What is exception propagation?

📊 Importance: ★★★★★ (Very High)

👉 Answer:

“Exception propagation means an exception is passed from one method to another until it is handled or reaches the JVM.”

? Can we write try without catch?

 Importance: ★★★★★ (High)

👉 Answer:

“Yes, we can use try without catch if we have a finally block or use try-with-resources.”

? Can we write catch without try?

 Importance: ★★★★★ (Very High)

👉 Answer:

“No, catch block cannot exist without a try block.”

? Can we have multiple finally blocks?

 Importance: ★★★★★ (High)

👉 Answer:

“No, only one finally block is allowed per try block.”

? Difference between printStackTrace(), getMessage(), toString()

 Importance: ★★★★★ (Very High)

👉 Already covered — very important revision point.

? What is Pipe Operator (|) in Java?

 Importance: ★★★★★ (High)

👉 Interview-ready answer:

“The pipe operator (|) in Java is used in multi-catch blocks to handle multiple exceptions in a single catch statement. It allows writing cleaner and shorter code when the handling logic for different exceptions is the same.”

👉 Example:

```
try {  
    int a = 10 / 0;  
} catch (ArithmeticException | NullPointerException e) {  
    System.out.println("Exception handled");  
}
```

Final Interview Closing Answer

“The pipe operator is used in multi-catch to handle multiple exceptions in a single catch block, making the code more concise.”

What is Try-With-Resources?

 Importance: ★★★★★ (Very High)

 Interview-ready answer (clean English):

“Try-with-resources is a feature in Java used to automatically close resources like files, streams, or database connections. It ensures that resources are closed after use without needing a finally block, making the code cleaner and safer.”

 Example:

```
import java.io.FileReader;  
  
class Test {  
    public static void main(String[] args) {  
        try (FileReader fr = new FileReader("file.txt")) {  
            int data = fr.read();  
            System.out.println(data);  
        } catch (Exception e) {  
            System.out.println("Exception handled");  
        }  
    }  
}
```

Where is the file getting closed in try-with-resources?

 Importance: ★★★★★ (Very High)

 Interview-ready answer (simple English):

“In try-with-resources, the resource is closed automatically by the JVM after the try block finishes execution. We do not see the close () call in the code because the compiler internally adds it. Any class used in try-with-resources must implement the AutoCloseable interface, which has the close () method. So, after the try block ends, the JVM automatically calls the close () method of the resource.”

👉 Understanding your example:

```
try (FileReader fr = new FileReader("file.txt")) {  
    int data = fr.read();  
    System.out.println(data);  
}
```

👉 What happens internally:

```
FileReader fr = new FileReader("file.txt");  
try {  
    int data = fr.read();  
    System.out.println(data);  
} finally {  
    if (fr != null) {  
        fr.close(); // automatically added by compiler  
    }  
}
```

❓ What is try in Java?

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (simple & clean):

“The try block in Java is used to write code that may cause an exception. It is always followed by either a catch block or a finally block. If an exception occurs inside the try block, it is handled by the catch block, preventing the program from crashing.”

❓ What is catch in Java?

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (simple & clean):

“The catch block in Java is used to handle exceptions that occur in the try block. It catches the exception object and allows the program to handle the error instead of terminating.”

? What is throw in Java?

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (simple & clean):

“The throw keyword in Java is used to explicitly create and throw an exception. It allows us to manually generate an exception when a specific condition occurs.”

? What is throws in Java?

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (simple & clean):

“The throws keyword in Java is used in a method declaration to indicate that the method may throw one or more exceptions. It informs the caller to handle those exceptions.”

? What is String in Java?

 Importance: ★★★★★ (Very High)

👉 Final Interview Answer (your style, clear & complete):

“String is a non-primitive data type in Java that represents a sequence of characters. It is an object of the String class and is immutable, meaning its value cannot be changed once created. The reference variable is stored in stack memory, while the actual String object is stored in heap memory.

Java also uses a special memory area called the String Constant Pool, which is a part of heap. When we create a String using a literal, the object is stored in the String Constant Pool, and duplicate values are avoided. If the same literal already exists, the reference points to the existing object.

When we use the new keyword, a new object is always created in heap memory, even if the same value exists in the String Constant Pool. In this case, the reference variable points to the heap object.

Garbage collection for String objects is handled by JVM, and objects are removed only when they are no longer referenced and the JVM decides to run garbage collection.”

what will happen when we create string using new keyword but not giving any literals.

Interview Answer

“When we create a String using new String() without passing any literal, Java creates an empty string object in heap memory and not in the scp. Its value is "" with length 0, and after that when we initialize String s1="" then this stored in scp. After reassignment, the reference points to the String Constant Pool object, and the previous heap object becomes eligible for garbage collection.”

? What is String Constant Pool (SCP)?

 Importance: ★ ★ ★ ★ ★ (Very High)

 Interview-ready answer (your style, clear):

“String Constant Pool is a special part of heap memory where Java stores String literals. It is used to avoid duplicate objects and save memory. When a String is created using a literal, JVM first checks if the same value already exists in the pool. If it exists, it reuses the existing object; otherwise, it creates a new one.”

String constant pool or String literals pool with Example

Clean & Correct Explanation with example

“When we write String s1 = new String ("Yasif");, two objects are created. One object is created in the String Constant Pool (SCP) if it does not already exist, and another object is created in the heap using the new keyword. The reference variable s1 stored in stack points to the object created in the heap, not the SCP.

When we create String s2 = new String ("Yasif");, only one new object is created in the heap, because the literal 'Yasif' already exists in the SCP. So JVM does not create another SCP object.

When we write String s3 = "Yasif";, no new object is created if the literal already exists in the SCP. The reference variable s3 simply points to the existing object in the SCP.

Similarly, String s4 = "Ashh"; creates a new object in SCP if it does not exist, and String s5 = "Ashh"; will point to the same SCP object without creating a new one.

Also, objects in the SCP are managed by JVM and are not immediately eligible for garbage collection, unlike heap objects created using new.”

? Full Syntax of String Class (Java)

 Importance: ★ ★ ★ ★ (High)

 Interview-ready answer (your style, clean):

“Yes, the String class is declared as a final class and it implements some interfaces. The correct full syntax is:”

```
public final class String extends Object
    implements Serializable, Comparable<String>, CharSequence
```

 Use of Each Interface:

◇ 1. Serializable

“Used to convert String object into byte stream for storage or transfer (like file or network).”

◇ 2. Comparable<String>

“Used to compare two strings using compareTo () method, mainly for sorting.”

"apple". compareTo("banana"); // comparison

◇ 3. CharSequence

“Represents a sequence of characters and allows methods like length (), charAt (), subSequence ().”

? Why is String immutable in Java?

 Importance: ★ ★ ★ ★ ★ (Very High)

 Interview-ready answer (your style, clean):

“String is immutable in Java to provide security, memory efficiency, and thread safety. Since String objects cannot be changed once created, they can be safely shared between multiple threads without synchronization. Immutability also allows Java to use the String Constant Pool, which avoids creating duplicate objects and saves memory.”

? Why is String final?

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (simple & clean):

“String is declared as final so that no class can extend it and change its behavior. This helps maintain immutability, security, and consistency of String objects.”

? Difference between final and immutability

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (simple & clean):

“Final is a keyword that restricts modification, while immutability is a property where the object’s state cannot change after creation. A final variable cannot be reassigned, but the object it refers to can still change. In contrast, an immutable object cannot be changed at all once it is created.”

? String features (Security, Class Loading, Synchronization, Concurrency, Memory)

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“String provides important features like security, class loading support, thread safety, and memory management. Because String is immutable, its value cannot be changed, which makes it secure for sensitive data like file paths, URLs, and class names.

In class loading, String is used to identify classes, and immutability ensures the correct class is loaded without modification.

String is thread-safe because it is immutable, so multiple threads can use the same object without synchronization issues. This also helps in concurrency as no extra locking is required.

For memory management, Java uses the String Constant Pool to reuse objects and avoid duplicates, which improves performance and saves memory.

It also provides many built-in methods for string manipulation like `length ()`, `charAt ()`, `substring ()`, and `compareTo ()`. Additionally, it implements interfaces like `Serializable`, `Comparable`, and `CharSequence`, which make it flexible and powerful.”

? Methods of Object Class (with use)

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“The Object class is the root class of Java, and all classes inherit its methods. These methods are used for comparison, synchronization, cloning, and basic object operations.”

🔥 Important Methods & Their Use

◆ equals (Object obj)

“Compares two objects for equality.”

◆ hashCode()

“Returns a hash value of the object, used in hashing (HashMap, etc.).”

◆ toString ()

“The toString () method is used to convert an object into a readable string representation.”

System.out.println(obj); // calls toString ()

◆ getClass ()

“Returns runtime class information of the object.”

◆ clone ()

“Creates and returns a copy of the object.”

◆ finalize () (Deprecated)

“Called by garbage collector before object is destroyed.”

🔗 Thread / Synchronization Methods

◆ wait ()

“Makes current thread wait until notified.”

◆ **wait(long timeout)**

“Waits for specified time.”

◆ **wait(long timeout, int nanos)**

“Waits for time with nanoseconds precision.”

◆ **notify()**

“Wakes up one waiting thread.”

◆ **notifyAll()**

“Wakes up all waiting threads.”

? **Difference between == and equals()**

 **Importance:** ★★★★★ (Very High)

 **Interview-ready answer (simple & clean):**

“The == operator is used to compare references, meaning it checks whether two variables point to the same memory location. The equals () method is used to compare the actual content or values of objects. For primitive types, == compares values, but for objects, it compares references unless equals () is overridden.”

Quick Difference:

Feature	==	equals()
Use	Reference comparison	Content comparison
Works on	Primitives & Objects	Objects only
Default behavior	Compares address	Compares reference (can be overridden)
String behavior	compares reference	compares value

? Examples of ==, Object.equals(), and String.equals()

 Importance: ★★★★★ (Very High)

💧 1. == (Reference Comparison)

```
String s1 = new String("Hello");
String s2 = new String("Hello");

System.out.println(s1 == s2); // false
```

👉 Different objects → different memory → **false**

💧 2. Object.equals() (Default - Reference Comparison)

```
class Test {}

Test t1 = new Test();
Test t2 = new Test();

System.out.println(t1.equals(t2)); // false
```

👉 equals() not overridden → behaves like ==

💧 3. String.equals() (Content Comparison)

```
String s1 = new String("Hello");
String s2 = new String("Hello");

System.out.println(s1.equals(s2)); // true
```

👉 String overrides equals() → compares **content**

? How do we know which equals() is used?

 Importance: ★★★★★ (Very High)

 Short answer (your style):

“It depends on the object type. If the class overrides equals(), then that method is used for content comparison. If not, the default Object class equals() is used, which compares reference.”

Why is char array is preferred over String for storing password?

 Clean & Correct Answer (your style):

“Char array is preferred over String for storing passwords because String objects are immutable. Once a password is stored in a String, it cannot be changed and stays in memory until garbage collection, which is not predictable. Also, String literals may be stored in the String Constant Pool, increasing security risk.

With a char array, we can manually clear or overwrite the data after use, so the password is removed from memory immediately. This makes char array more secure than String for sensitive data like passwords.”

? What is StringBuffer in Java?

 Importance: ★★★★★ (Very High)

 Interview-ready answer (simple & clean):

“StringBuffer is a class in Java used to create mutable strings, meaning its value can be changed after creation. It is thread-safe because its methods are synchronized, so it can be safely used in multi-threaded environments.”

? What is StringBuilder in Java?

 Importance: ★★★★★ (Very High)

 Interview-ready answer (simple & clean):

“StringBuilder is a class in Java used to create mutable strings, meaning its value can be changed after creation. It is not thread-safe because its methods are not synchronized, but it is faster than StringBuffer. It is mainly used in single-threaded environments where performance is important.”

? Difference between String, StringBuffer, and StringBuilder

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“String, StringBuffer, and StringBuilder are used to handle text in Java, but they differ in mutability and thread safety. String is immutable, so its value cannot be changed. StringBuffer and StringBuilder are mutable, so their values can be modified without creating new objects. StringBuffer is thread-safe because its methods are synchronized, while StringBuilder is not thread-safe but faster.”

💧 Quick Difference:

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Thread Safety	Yes	Yes (synchronized)	No
Performance	Slow	Medium	Fast
Memory	More (new object)	Less	Less

🔥 Remaining Important String Questions

? What is intern() method?

 Importance: ★★★★★

“intern() is used to move a String object to the String Constant Pool or return reference of existing SCP object.”

? How many objects are created? (Tricky)

 Importance: ★★★★★

String s = new String("Hello");

“One in heap and one in SCP (if not already present).”

? Difference between String and char[]

 Importance: ★★★★★

“String is immutable, char[] is mutable and more secure for passwords.”

? Difference between Multitasking, Multiprocessing, and Multithreading

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“Multitasking, multiprocessing, and multithreading are concepts used to execute multiple tasks, but they differ in how tasks are handled. Multitasking means running multiple tasks at the same time using a single CPU by switching between them. Multiprocessing uses multiple CPUs or cores to execute processes in parallel. Multithreading is a lightweight process where multiple threads run within a single process and share the same memory.”

🔥 Quick Difference:

Feature	Multitasking	Multiprocessing	Multithreading
Definition	Multiple tasks on single CPU	Multiple processes on multiple CPUs	Multiple threads in one process
CPU Usage	Single CPU (time-sharing)	Multiple CPUs/cores	Single or multiple cores
Memory	Separate memory per process	Separate memory per process	Shared memory
Performance	Moderate	High	High (lightweight)
Complexity	Medium	High	Medium

? Difference between Process and Thread

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“A process is an independent program in execution with its own memory space, while a thread is a lightweight unit of execution within a process that shares the same memory. Processes are heavy-weight and isolated, whereas threads are light-weight and faster for communication.”

🔗 Process vs Thread (Interview Closing Answer)

Definition (💧💧 Very High Importance)

A process is an independent program with its own memory, while a thread is a smaller unit of a process that shares memory with other threads.

Context Switching (💧 High Importance)

Process switching is slower because it involves switching memory and resources, while thread switching is faster since threads share the same memory.

Communication (💧 High Importance)

Processes communicate using IPC methods like pipes or sockets, which are slower, while threads communicate directly through shared memory, which is faster.

Address Space (💧💧 Very High Importance)

Each process has its own separate address space, whereas threads share the same address space within a process.

Dependency (💧 Medium Importance)

Processes are independent of each other, while threads are dependent on the process and other threads.

Synchronization (💧💧 Very High Importance)

Processes need less synchronization, but threads require proper synchronization (like locks) to avoid issues like race conditions.

? How to Create Thread in Java

📖 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“In Java, a thread can be created in two main ways: by extending the Thread class or by implementing the Runnable interface. Extending Thread is simple but limits inheritance, while implementing Runnable is preferred because it allows better design and flexibility.”

⚡ Key Point (Important for Interview)

- `start()` → creates a new thread and calls `run()`
- `run()` → contains the actual logic

? Thread Lifecycle in Java (Detailed Closing Answer)

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, detailed & clean):

“Thread lifecycle in Java defines the different states that a thread goes through during its execution. Initially, a thread is in the new state when it is created but not started. When the `start()` method is called, it moves to the Runnable state, where it is ready to run and waiting for CPU scheduling. Once the CPU is assigned, it enters the Running state and starts executing the `run()` method.

During execution, a thread may enter the Waiting or Blocked state if it needs to wait for a resource, another thread, or due to methods like `sleep()` or `wait()`. After the waiting condition is resolved, it moves back to the Runnable state and can again be scheduled for execution. Finally, once the execution is completed, the thread enters the Terminated state, where it stops permanently.

So overall, a thread transitions through states like New, Runnable, Running, Waiting/Blocked, and Terminated based on its execution flow and system scheduling.”

? Is Blocked/Waiting State also called Non-Runnable State?

 Importance: ★★★★★☆ (High)

👉 Interview-ready answer (your style, clean):

“Yes, Blocked and Waiting states are generally considered non-runnable states because the thread is not eligible to run until a specific condition is met, such as acquiring a lock or receiving a signal from another thread.”

? Different Ways of Executing Tasks using Threads in Java

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“In Java, tasks can be executed using threads in different ways depending on how many tasks and threads are involved. These approaches define how concurrency and performance are achieved in an application.”

Quick Difference:

Case	Description
Single Task – Single Thread	One thread executes one task (no concurrency)
Single Task – Multiple Threads	Multiple threads execute the same task concurrently
Multiple Tasks – Single Thread	One thread executes multiple tasks sequentially
Multiple Tasks – Multiple Threads	Multiple threads execute multiple tasks concurrently

Final Interview Closing Answer (your style):

“Threads can execute tasks in four ways: single task by single thread, single task by multiple threads, multiple tasks by single thread, and multiple tasks by multiple threads, where the last approach provides maximum concurrency and performance.”

Daemon Thread in Java

Final Interview Closing Answer (Daemon Thread - Simple & Clean)

“A daemon thread in Java is a background thread that supports main tasks of the program. It is used for things like garbage collection, logging, and monitoring. We can make a thread daemon using `setDaemon(true)` before starting it, and check it using `isDaemon()`. When all user threads finish, the JVM automatically stops daemon threads. So, daemon threads are used for background work and do not keep the program running.”

Important Methods:

Method	Description
<code>setDaemon(true)</code>	Converts a thread into daemon thread
<code>isDaemon()</code>	Checks whether thread is daemon or not

⚠️ Must call `setDaemon(true)` **before** `start()`, otherwise it throws exception. `IllegalThreadStateException`.

```
class MyDaemon extends Thread {
    public void run() {
        while (true) {
            System.out.println("Daemon thread running...");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyDaemon t1 = new MyDaemon();
        t1.setDaemon(true); // setting daemon thread
        t1.start();

        System.out.println("Main thread finished");
    }
}
```

Priority Thread methods in the notes.

? Thread Priority in Java

📊 Importance: ★★★★★ (High)

👉 Interview-ready answer (your style, clean):

“Thread priority in Java is used to indicate the importance of a thread. The JVM assigns CPU time to threads based on their priority. Priority values range from 1 to 10, where 1 is the lowest and 10 is the highest, and the default priority is 5. A thread inherits its priority from its parent thread. If a value outside 1 to 10 is set, it throws an `IllegalArgumentException`. If multiple threads have the same priority, the JVM decides which one to execute. Also, thread priority behavior may vary depending on the operating system.”

🔥 Important Methods:

Method	Description
setPriority(int n)	Sets thread priority (1–10)
getPriority()	Returns current priority

Example Program:

```
class MyThread extends Thread {
    public void run() {
        System.out.println(Thread.currentThread().getName() + " Priority: " + Thread.currentThread().getPriority());
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();

        t1.setPriority(Thread.MIN_PRIORITY); // 1
        t2.setPriority(Thread.MAX_PRIORITY); // 10

        t1.start();
        t2.start();
    }
}
```

```
Thread().getName() + " Priority: " + Thread.currentThread().getPriority());

) {

Y); // 1
Y); // 10
```

Final Interview Closing Answer (your style):

“Thread priority in Java ranges from 1 to 10 and is used by the JVM to decide execution order, but actual execution may depend on the operating system.”

? What is the use of sleep() method in Java?

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“The sleep() method in Java is used to pause the execution of a thread for a specified amount of time. During this time, the thread goes into a timed waiting state and does not use CPU, allowing other threads to execute.”

? yield() and join() Methods in Java

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“In Java, yield() and join() are thread control methods. The yield() method pauses the current thread and gives a chance to other threads of the same priority to execute, while join() makes one thread wait until another thread completes its execution.”

? Difference between sleep(), yield(), and join()

 Importance: ★★★★★ (Very High)

🔥 Quick Difference:

Feature	sleep()	yield()	join()
Purpose	Pause thread for fixed time	Give chance to other threads	Wait for another thread
Control	Time-based	Scheduler-based (hint)	Depends on another thread
State	Timed Waiting	Runnable	Waiting
Exception	Yes (InterruptedException)	No	Yes (InterruptedException)

? interrupt() Method in Java

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“The `interrupt()` method in Java is used to interrupt a thread that is in a waiting or sleeping state. It does not stop the thread immediately, but signals it to stop by setting an interrupt flag. If the thread is in methods like `sleep()` or `wait()`, it throws an `InterruptedException`.”

🔥 Key Points:

Feature	Description
Purpose	Interrupt a thread
Behavior	Sets interrupt flag
Exception	Throws <code>InterruptedException</code> (if sleeping/waiting)
Stop Thread	Does not force stop, just signals

? Types of Synchronization: Process & Thread Synchronization

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“Synchronization in Java is mainly of two types: process synchronization and thread synchronization. Process synchronization is used to manage multiple independent processes, while thread synchronization is used to control multiple threads within the same process that share memory.”

💧 Thread Synchronization Types:

Type	Description
Mutual Exclusion	Only one thread accesses critical section at a time
Cooperation (Inter-thread Communication)	Threads communicate and coordinate using methods like <code>wait()</code> , <code>notify()</code> , and <code>notifyAll()</code>

? Types of Synchronization in Java (Method, Block, Static)

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, clean):

“In Java, synchronization can be applied in three ways: synchronized method, synchronized block, and static synchronization. These are used to control access to shared resources and ensure that only one thread executes critical code at a time.”

 Quick Difference:

Type	Description
Synchronized Method	Locks entire method
Synchronized Block	Locks only a specific part of code
Static Synchronization	Locks class-level (shared among all objects)

? Synchronized Method in Java

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, simple & clean):

“A synchronized method in Java is used to control access to a method so that only one thread can execute it at a time. It helps in preventing data inconsistency when multiple threads access shared data.”

? Synchronized Block in Java

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, simple & clean):

“A synchronized block in Java is used to lock only a specific part of the code instead of the entire method. It allows better performance because only the critical section is locked, not the whole method.”

 How It Works:

- Uses `synchronized(object)`
- Locks only the **given object**

- Only one thread can enter that block at a time
-

🔥 Syntax:

```
synchronized(object) {  
    // critical section  
}
```

? Static Synchronized in Java

📖 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, simple & clean):

“Static synchronized in Java is used to lock class-level resources so that only one thread can access the static method at a time. It locks the Class object instead of the instance, meaning the lock is shared across all objects of the class.”

🔥 How It Works:

- Uses static synchronized keyword
 - Locks the **Class object (class-level lock)**
 - Shared across **all instances**
-

🔥 Syntax:

```
static synchronized void methodName() {  
    // critical section  
}
```

🔥 1. Synchronized Method

```
class Counter {  
    int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
}
```

- Lock is on **current object (this)**

- Only one thread can execute method at a time
-

🔥 2. Synchronized Block

```
class Counter {  
    int count = 0;  
  
    void increment() {  
        synchronized(this) {  
            count++;  
        }  
    }  
}
```

- Lock is on **specific object**
 - More efficient than method (only critical section locked)
-

🔥 3. Static Synchronization

```
class Counter {  
    static int count = 0;  
  
    static synchronized void increment() {  
        count++;  
    }  
}
```

⚡ Key Points:

- Prevents **race condition**
- Ensures **thread safety**
- Too much synchronization → reduces performance

? Inter-Thread Communication in Java

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“Inter-thread communication in Java is a mechanism that allows threads to communicate and coordinate with each other. It is used to avoid busy waiting and to ensure proper execution order between threads.”

💧 Achieved by 3 Methods:

Method	Description
--------	-------------

wait()	Makes thread wait until notified
--------	----------------------------------

notify()	Wakes up one waiting thread
----------	-----------------------------

notifyAll()	Wakes up all waiting threads
-------------	------------------------------

⚡ Key Points:

- Defined in **Object class**
- Must be used inside **synchronized block/method**
- Helps in **thread cooperation**

? Remaining Important Thread Topics for Interview

📊 Importance: ★★★★★ (Very High)

🔥 1. Deadlock

👉 Interview-ready:

“Deadlock is a situation where two or more threads are blocked forever, waiting for each other’s resources.”

✓ Must know:

- Causes (circular wait)
 - Prevention (lock ordering)
-

🔥 2. Race Condition

 Interview-ready:

“Race condition occurs when multiple threads access shared data at the same time and the final result depends on the execution order.”

✓ **Solved using:**

- Synchronization
-

 3. Thread Safety

 Interview-ready:

“Thread safety means that shared data is accessed safely by multiple threads without causing inconsistency.”

✓ **Achieved using:**

- Synchronization
 - Immutable objects
-

 4. Volatile Keyword

 Interview-ready:

“The volatile keyword ensures visibility of changes to variables across threads, meaning one thread’s update is immediately visible to others.”

 5. Executor Framework (Very Important Modern Concept)

 Interview-ready:

“Executor framework is used to manage thread creation and execution using thread pools, improving performance and resource management.”

 6. Callable & Future

 Interview-ready:

“Callable is similar to Runnable but can return a result and throw exceptions, and Future is used to get the result of Callable.”

🔥 7. Difference: Runnable vs Callable

👉 Short line:

“Runnable does not return result, while Callable returns result and can throw exception.”

? Difference between interrupt(), isInterrupted(), and interrupted()

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean & detailed):

“In Java, these three methods are used to handle thread interruption. The `interrupt()` method is used to send a signal to a thread that it should stop or handle interruption. The `isInterrupted()` method checks whether a thread has been interrupted without changing its status. The `interrupted()` method checks the interrupt status of the current thread and also clears it, meaning after calling it once, the status becomes false.”

🔥 Key Difference:

Method	Type	Behavior
<code>interrupt()</code>	Instance	Sets interrupt flag
<code>isInterrupted()</code>	Instance	Checks flag (does NOT clear)
<code>interrupted()</code>	Static	Checks flag and clears it

🔥 Example (All in One Code):

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Thread started");

        while (!Thread.currentThread().isInterrupted()) {
            System.out.println("Running...");

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println("Interrupted during sleep");

                // Check using interrupted() (this will clear flag)
                System.out.println("interrupted(): " + Thread.interrupted());

                // Check again using isInterrupted()
                System.out.println("isInterrupted(): " + Thread.currentThread().isInterrupted());

                break;
            }
        }
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        MyThread t1 = new MyThread();
        t1.start();

        Thread.sleep(2000);

        t1.interrupt(); // sending interrupt signal
    }
}

```

⚡ What Happens Here:

- `interrupt()` → sends signal to thread
- If thread is sleeping → `InterruptedException` occurs
- `interrupted()` → checks and **clears** flag
- `isInterrupted()` → checks flag but **does not clear**

? Deadlock in Java

📖 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“Deadlock is a situation in Java where two or more threads are blocked forever, waiting for each other to release resources. As a result, none of the threads can proceed, and the program gets stuck.”

🔥 Example Situation:

- Thread 1 holds **Lock A** and waits for **Lock B**
 - Thread 2 holds **Lock B** and waits for **Lock A**
- 👉 Both are waiting → **Deadlock occurs**
-

🔥 Example Program:

```
class Test {
    public static void main(String[] args) {
        final Object lock1 = new Object();
        final Object lock2 = new Object();

        Thread t1 = new Thread(() -> {
            synchronized (lock1) {
                System.out.println("Thread 1 locked lock1");
                try { Thread.sleep(100); } catch (Exception e) {}

                synchronized (lock2) {
                    System.out.println("Thread 1 locked lock2");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (lock2) {
                System.out.println("Thread 2 locked lock2");
                try { Thread.sleep(100); } catch (Exception e) {}
            }
        });
    }
}
```



```
Thread t2 = new Thread(() -> {
    synchronized (lock2) {
        System.out.println("Thread 2 locked lock2");
        try { Thread.sleep(100); } catch (Exception e) {}

        synchronized (lock1) {
            System.out.println("Thread 2 locked lock1");
        }
    }
});

t1.start();
t2.start();
}
```

⚠ What Happens:

- t1 locks lock1 and waits for lock2
- t2 locks lock2 and waits for lock1
- 👉 Both threads wait forever → **Program stuck**

🔥 Conditions for Deadlock (Important):

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

🔥 Prevention Techniques:

- Maintain **proper lock order**
 - Avoid **nested locks**
 - Use **tryLock()** (advanced)
-

? What is Collection Framework in Java?

 Importance: ★ ★ ★ ★ ★ (Very High)

 Interview-ready answer (your style, clean):

“The Collection Framework in Java is a set of classes and interfaces used to store, manage, and manipulate groups of objects efficiently. It provides ready-made data structures like List, Set, and Map, along with methods to perform operations like adding, removing, and searching elements.”

? What is Collection in Java?

 Importance: ★ ★ ★ ★ ★ (Very High)

 Interview-ready answer (your style, clean):

“A collection in Java is a group of objects treated as a single unit. It is used to store, retrieve, and manipulate multiple objects dynamically.”

? What is Collections in Java?

 Importance: ★ ★ ★ ★ ★ (Very High)

 Interview-ready answer (your style, clean):

“Collections in Java is a utility class that provides static methods to perform operations on collection objects, such as sorting, searching, and reversing.”

? Difference between Collection Framework, Collection, and Collections

 Importance: ★ ★ ★ ★ ★ (Very High)

 Interview-ready answer (your style, clean):

“Collection Framework is the complete set of classes and interfaces used to manage groups of objects. Collection is an interface in this framework that represents a group of objects, while Collections is a utility class that provides static methods to perform operations on collection objects.”

? What are Legacy Classes in Java?

 Importance: ★ ★ ★ ★ ☆ (High)

 Interview-ready answer (your style, clean):

“Legacy classes in Java are the old classes that were introduced before the Collection Framework. Later, they were modified to fit into the Collection Framework. These classes are synchronized by default and are considered outdated for most modern use cases.”

? What is List in Java?

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, clean):

“List in Java is an interface in the Collection Framework that represents an ordered collection of elements. It allows duplicate values and maintains insertion order, so elements can be accessed by index.”

? How can we get elements from a List in Java?

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, clean):

“Elements from a List in Java can be accessed using index-based methods like `get(index)`, or by using loops and iterators to traverse all elements.”

? What is Set in Java and how to get elements?

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, clean):

“Set in Java is an interface in the Collection Framework that represents a collection of unique elements. It does not allow duplicates and does not guarantee insertion order. Since Set does not support indexing, elements cannot be accessed using index and must be retrieved using iteration methods.”

? Difference between Array and LinkedList

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, clean):

“Array stores elements in contiguous memory locations, while LinkedList stores elements in nodes where each node contains data and a reference to the next node. Arrays have fixed size and provide

fast access using index, whereas LinkedList has dynamic size but slower access. However, insertion and deletion are faster in LinkedList compared to Array.”

? Difference between Array and List

 Importance: ★ ★ ★ ★ ★ (Very High)

 Interview-ready answer (your style, clean):

“Array is a fixed-size data structure that can store elements of the same type, while List is a part of the Collection Framework that provides dynamic size and more flexibility. Arrays support both primitive and object types, whereas List can store only objects. List also provides many built-in methods for easy manipulation of data.”

? Difference between List and Set

 Importance: ★ ★ ★ ★ ★ (Very High)

 Interview-ready answer (your style, clean):

“List and Set are both part of the Collection Framework, but List allows duplicate elements and maintains insertion order, while Set does not allow duplicate elements and does not guarantee order.”

? Difference between Array and Collection Framework

 Importance: ★ ★ ★ ★ ★ (Very High)

 Interview-ready answer (your style, clean):

“Array is a basic data structure used to store fixed-size elements of the same type, while the Collection Framework is a set of classes and interfaces that provide dynamic data structures like List, Set, and Map with many built-in methods for easy data handling.”

? Important Collection Interface Methods (Interview Focus)

 Importance: ★ ★ ★ ★ ★ (Very High)

 Interview-ready answer (your style, clean):

“We don’t need to remember all methods, but some common methods of the Collection interface are important for interviews.”

Must-Remember Methods (Collection Interface)

```
add(E e)      // add element
remove(Object o) // remove element
contains(Object o) // check element exists
size()        // number of elements
isEmpty()      // check empty or not
clear()        // remove all elements
iterator()     // get iterator
```

List Specific (Important)

```
get(int index) // access element
set(int index, E e) // update element
```

Set Specific (Important)

```
add()      // no duplicate allowed
👉 (same as Collection, but concept important)
```

Map Important Methods

```
put(K key, V value)
get(Object key)
remove(Object key)
containsKey(Object key)
keySet()
values()
```

? How does Set store data in Java?

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, clean):

“Set is used to store unique elements. When we add an element, first its hashCode() method is called to generate a hash value. This hash value is then converted into a bucket index to decide where to store the element in the internal array.

Then Java checks that bucket position. If the bucket is empty, the element is stored directly. If the bucket is not empty, then equals() is used to compare the new element with existing elements in that bucket. If a duplicate is found, it is not stored; otherwise, it is stored in the same bucket using a linked list or tree (in newer versions).

Internally, HashSet uses HashMap where elements are stored as keys and a dummy object is stored as value.”

? Internal Working of HashSet

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, clean)

“HashSet stores elements using hashing. When we add an element, first its hashCode() is generated, then it is converted into a bucket index, which decides the position in the internal array. If the bucket is empty, the element is stored directly. If not, equals() is used to check duplicates. If the element is already present, it is not stored; otherwise, it is added in the same bucket using a linked list or tree. Internally, HashSet uses HashMap to store elements as keys with a dummy value.”

? How Hashtable works in Java

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, clean & detailed):

“Hashtable is used to store data in key-value pairs using hashing. When we add a key-value pair, first the key’s hashCode() method is called to generate a hash value. This hash value is then converted

into a bucket index, and that index itself is the position where the entry will be stored in the internal array.

After that, Java directly goes to that bucket index. If the bucket is empty, the key-value pair is stored there. If the bucket is not empty, then `equals()` is used to compare the new key with existing keys in that same bucket. If the same key is found, its value is updated; otherwise, the new entry is stored in the same bucket using a linked list (collision handling).

Hashtable is synchronized, so all its methods are thread-safe, but this makes it slower compared to HashMap.”

? Difference between HashMap and Hashtable

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“HashMap and Hashtable both store data in key-value pairs using hashing, but HashMap is not synchronized and allows one null key and multiple null values, while Hashtable is synchronized and does not allow any null key or value.”

? Difference between Set and Hashtable

 Importance: ★★★★★ (High)

👉 Interview-ready answer (your style, clean):

“Set is used to store only unique elements, while Hashtable is used to store key-value pairs. Set does not allow duplicates, whereas Hashtable does not allow duplicate keys but allows different values for different keys. Hashtable is synchronized and thread-safe, while Set implementations like HashSet are not synchronized.”

? What is Iterator?

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“Iterator is an interface in Java used to traverse elements of a collection one by one. It provides methods like `hasNext()`, `next()`, and `remove()` to iterate and manipulate elements safely.”

? What is ListIterator?

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“ListIterator is an interface in Java used to traverse elements of a List in both forward and backward direction. It provides additional methods compared to Iterator, such as previous(), hasPrevious(), add(), and set(), which allow modification of elements during traversal.”

? What is Enumeration?

📊 Importance: ★★★★★ (High)

👉 Interview-ready answer (your style, clean):

“Enumeration is an interface in Java used to traverse elements of legacy classes like Vector and Hashtable. It allows only forward traversal, can read only and does not support modification of elements.”

? What is Cursor in Java?

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“Cursor is an interface used to traverse elements of a collection one by one. It helps in accessing data sequentially without exposing the internal structure of the collection.”

? Iterator vs ListIterator

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“Iterator and ListIterator are used to traverse collection elements. Iterator works for all collections and supports only forward traversal with read and remove operations. ListIterator works only for List and supports both forward and backward traversal with additional operations like add, set, and remove.”

🔥 Comparison

Feature	Iterator	ListIterator
Works on	All collections	Only List

Feature	Iterator	ListIterator
Direction	Forward only	Forward + Backward
Methods	hasNext(), next(), remove()	hasNext(), next(), hasPrevious(), previous(), add(), set(), remove()
Operations	Read + Remove	Read + Add + Update + Remove
Flexibility	Less	More

? What is ArrayList.

 Importance: ★★★★★ (Very High)

“ArrayList is a class that implements the List interface. It is based on a dynamic (growable and resizable) array. It is index-based, allows duplicate elements, and also allows null values. It maintains insertion order but does not follow sorting order. It is not synchronized, so it is not thread-safe, but it is faster due to no synchronization overhead. It usually stores same type of data when generics are used. And can store different data type when not use generics.”

? Constructors of ArrayList

 Importance: ★★★★★ (High)

👉 Interview-ready answer (your style, clean):

“ArrayList provides constructors to create a list with default size, with initial capacity, or by copying another collection.”

🔥 Constructors

```
ArrayList()           // default (capacity 10)
ArrayList(int capacity) // custom initial capacity
ArrayList(Collection c) // create from another collection
```

? Important Methods of ArrayList

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style):

“ArrayList provides many methods to add, remove, access, and manipulate elements.”

🔥 Must-Remember Methods

```
add(E e)           // add element
add(int index, E e) // add at index
get(int index)      // get element
set(int index, E e) // update element
remove(int index)   // remove by index
remove(Object o)    // remove by value
size()              // number of elements
isEmpty()           // check empty
clear()             // remove all
contains(Object o)  // check element
```

🔥 Extra Useful Methods

```
indexOf(Object o)
lastIndexOf(Object o)
toArray()
iterator()
```

❓ What is linked list.

📊 Importance: ★★★★★ (Very High)

✅ Correct Answer (your style, clean):

“LinkedList is a class that implements the List interface and is based on a doubly linked list. It is not truly index-based because elements are stored as nodes connected by links. It can store different data types without generics, allows duplicate values including null, and maintains insertion order but does not follow sorting order. It is not synchronized, so it is not thread-safe, but it is faster for insertion and deletion operations.”

❓ Constructors of LinkedList

📊 Importance: ★★★★★ (High)

👉 Interview-ready answer (your style, clean):

“LinkedList provides constructors to create an empty list or to create a list from another collection.”

🔥 Constructors

```
LinkedList()           // empty linked list
LinkedList(Collection c) // create from another collection
```

? Important Methods of LinkedList

📖 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style):

“LinkedList provides methods to add, remove, and access elements, and also special methods for handling first and last elements.”

🔥 Must-Remember Methods

```
add(E e)           // add element
add(int index, E e) // add at index
get(int index)      // get element
set(int index, E e) // update element
remove(int index)   // remove by index
remove(Object o)    // remove by value
size()              // number of elements
isEmpty()           // check empty
clear()             // remove all
contains(Object o)  // check element
```

🔥 LinkedList Specific Methods (Very Important)

```

addFirst(E e)    // add at beginning
addLast(E e)     // add at end
getFirst()       // get first element
getLast()        // get last element
removeFirst()    // remove first
removeLast()     // remove last

```

ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Underlying Data Structure	Dynamic array	Doubly linked list
Index Access	True index-based ($O(1)$)	Node traversal ($O(n)$)
Insertion/Deletion	Slow in middle ($O(n)$)	Fast in middle ($O(1)$ if node known)
Memory	Less memory overhead	More memory (stores prev & next references)
Null & Duplicates	Allows both	Allows both
Thread-Safe	No (non-synchronized)	No (non-synchronized)
Use Case	Frequent access	Frequent insertion/deletion

? Difference between ArrayList and Vector

“ArrayList and Vector both are dynamic array implementations, but ArrayList is not synchronized and faster, while Vector is synchronized and thread-safe but slower. Also, Vector is a legacy class, whereas ArrayList is part of the modern Collection Framework.”

? What is Stack class in Collection Framework?

 Importance: ★★ ★ (Medium)

👉 Interview-ready answer (your style, clean):

“Stack is a legacy class in Java that extends Vector and is used to store elements in LIFO (Last In First Out) order. It means the last element added is the first one to be removed.”

🔥 Properties of Stack:

- ☒ Follows **LIFO principle**
 - ☒ Extends **Vector (legacy class)**
 - ☒ **Synchronized** (thread-safe)
 - ☒ Slower due to synchronization
 - ☒ Considered **outdated** (Deque preferred now)
-

🔥 Important Methods:

```
push(E e) // add element
pop()     // remove top element
peek()    // view top element
empty()   // check if empty
```

? What is HashSet in Java?

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“HashSet is a class that implements the Set interface and is present in the java.util package. Its underlying data structure is a hash table, and it is backed by a HashMap. It does not allow duplicate elements, allows one null value, does not maintain insertion or sorting order, and is not synchronized, so it is not thread-safe.”

🔥 Key Properties:

- ☒ No duplicate elements
 - ☒ No guaranteed order
 - ☒ Allows one null value
 - ☒ Not synchronized (not thread-safe)
 - ⚡ Fast performance (uses hashing)
-

🔥 Internal Working (short)

“HashSet uses hashCode() to find bucket index and equals() to check duplicates. Internally, it uses HashMap to store elements.”

🔥 Constructors

```
HashSet()                // default capacity (16)
HashSet(int initialCapacity) // custom capacity
HashSet(int initialCapacity, float loadFactor)
HashSet(Collection c)     // from another collection
```

🔥 Important Methods

```
add(E e)                // add element
remove(Object o)         // remove element
contains(Object o)       // check element
size()                  // number of elements
isEmpty()               // check empty
clear()                 // remove all
iterator()              // traverse elements
```

🎯 Final Interview Closing Answer (your style):

“HashSet is used to store unique elements using hashing, does not allow duplicates, and internally uses HashMap.”

❓ Difference between HashSet, LinkedHashSet, and TreeSet

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“HashSet, LinkedHashSet, and TreeSet are all implementations of Set interface, but they differ in ordering and internal working. HashSet does not maintain any order, LinkedHashSet maintains insertion order, and TreeSet stores elements in sorted order.”

🔥 Quick Difference:

Feature	HashSet	LinkedHashSet	TreeSet
Order	✗ No order	☑ Insertion order	☑ Sorted order
Underlying DS	Hash table	Hash table + Linked list	Red-Black Tree
Performance	Fastest	Slightly slower	Slowest
Null	☑ One allowed	☑ One allowed	✗ Not allowed
Duplicate	✗ Not allowed	✗ Not allowed	✗ Not allowed

? What is TreeSet.

 Importance: ★★★★★ (Very High)

☑ Clean & Correct Definition (your style)

“TreeSet is a class that implements the NavigableSet interface and indirectly implements SortedSet and Set. Its underlying data structure is a self-balancing binary search tree (Red-Black Tree). It does not maintain insertion order but stores elements in sorted order. It does not allow duplicates, does not allow null values, and is not synchronized, so it is not thread-safe.”

🔥 Small Corrections

- ✓ “balanced tree” → say **Red-Black Tree** (interview impact)
- ✓ “stores same data type” → due to **Comparable/Comparator**
- ✓ “not index-based” → correct

? Constructors of TreeSet

 Importance: ★★★★★

👉 Interview answer (your style):

“TreeSet provides constructors to create sorted set with natural or custom ordering.”

```
TreeSet()           // default (natural sorting)
TreeSet(Comparator comp) // custom sorting
```

```
TreeSet(Collection c)    // from collection
TreeSet(SortedSet s)    // from sorted set
```

? Important Methods of TreeSet

 Importance: ★★★★★

 Interview answer (your style):

“TreeSet provides methods for sorting, navigation, and element operations.”

```
add(E e)        // add element
remove(Object o) // remove element
contains(Object o) // check element
size()          // number of elements
first()         // first (smallest)
last()          // last (largest)
```

🔥 Navigable Methods (Very Important)

```
higher(E e) // next greater element
lower(E e)  // next smaller element
ceiling(E e) // >= element
floor(E e)  // <= element
pollFirst() // remove first
pollLast()  // remove last
```

? Comparable (Final Combined Answer)

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, clean & complete):

“Comparable is an interface used to define the natural sorting order of objects, and it is implemented inside the class. It provides the `compareTo()` method, which compares the current object with a specified object. If it returns a positive value, the current object is greater; if negative, it is smaller; and if zero, both are equal. In `TreeSet`, this result is used to decide position—negative goes left, positive goes right, and zero means duplicate, so it is not stored.”

 Final Interview Line

“Comparable is used for natural sorting using `compareTo()`, which returns positive, negative, or zero to decide order and duplicates.”

? Comparator (Final Combined Answer)

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, clean & complete):

“Comparator is an interface used to define custom sorting logic, and it is implemented outside the class. It provides the `compare()` method, which compares two objects. If it returns a positive value, first object is greater; if negative, it is smaller; and if zero, both are equal. In `TreeSet`, this result is used for ordering—negative goes left, positive goes right, and zero means duplicate, so it is not stored.”

🔥 Step-by-step Working

✅ Case 1: Using Comparable (default)

```
TreeSet<Integer> set = new TreeSet<>();
set.add(30);
set.add(10);
set.add(20);
```

 Internally:

- Uses `compareTo()`
- Stores in sorted order → 10, 20, 30

✅ Case 2: Using Comparator (custom)

```
TreeSet<Integer> set = new TreeSet<>((a, b) -> b - a);
```

 Internally:

- Uses `compare()`
- Custom sorting → 30, 20, 10

? What is Map in Java?

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean & complete):

“Map is an interface that does not extend the Collection interface. It stores data in key-value pairs where keys are unique and values can be duplicated. The behavior of null values and ordering depends on the implementation. For example, HashMap allows one null key and multiple null values and does not maintain order.”

✗ Corrections

- ✗ “can store single null key and multiple null values”
👉 **Not for all Map implementations** (only for **HashMap**)
 - ✗ “does not follow insertion and sorting order”
👉 Depends on implementation:
 - HashMap → no order
 - LinkedHashMap → insertion order
 - TreeMap → sorted order
-

🔥 Common Implementations

- **HashMap** → no order, fastest
 - **LinkedHashMap** → maintains insertion order
 - **TreeMap** → sorted order
-

🔥 Important Methods

```
put(K key, V value)      // add/update
get(Object key)          // get value
remove(Object key)       // remove entry
containsKey(Object key)  // check key
containsValue(Object v)  // check value
size()                  // number of entries
isEmpty()               // check empty
keySet()                // get all keys
values()                // get all values
entrySet()              // key-value pairs
```

Final Interview Closing Answer (your style)

“Map is used to store key-value pairs where keys are unique, and it provides methods like put and get for data handling.”

How Map Works (Short Logic)

Interview-style explanation:

“Map uses hashing internally. When we put a key-value pair, hashCode() of the key is generated, converted into a bucket index, and stored. If the same key comes again, equals() is used and the value is updated.”

What is HashMap

 Importance: ★★★★★ (Very High)

Clean & Correct Definition (your style)

“HashMap is a class that implements the Map interface. Its underlying data structure is a hash table. It stores data in key-value pairs where keys are unique and values can be duplicated. It allows one null key and multiple null values. It does not maintain insertion or sorting order and is not synchronized, so it is not thread-safe.”

Small Corrections

- ✗ “store different data type”
 ☒ Only without generics (not recommended)
- ✗ “retrieve using set and iterator only”
 ☒ There are **multiple ways** (explained below)

? What is Load Factor

 Importance: ★★★★★ (Very Important)

👉 Interview answer (your style):

“Load factor defines how full the HashMap can get before resizing. Default value is 0.75. When the number of elements exceeds capacity × load factor, the HashMap increases its size (rehashing).”

? Constructors of HashMap

 Importance: ★★★★★

`HashMap()` // default (capacity 16, load factor 0.75)

`HashMap(int initialCapacity)` // custom capacity

`HashMap(int initialCapacity, float loadFactor)`

`HashMap(Map m)` // from another map

? Important Methods

 Importance: ★★★★★

```
put(K key, V value)      // add/update
get(Object key)          // get value
remove(Object key)       // remove
containsKey(Object key)  // check key
containsValue(Object v)  // check value
size()                  // number of entries
isEmpty()               // check empty
clear()                 // remove all
```

? Ways to Retrieve Data

✅ 1. Using `keySet()`

```
for (Integer key : map.keySet()) {  
    System.out.println(key + " " + map.get(key));  
}
```

✅ 2. Using entrySet() (Best way)

```
for (Map.Entry<Integer, String> e : map.entrySet()) {  
    System.out.println(e.getKey() + " " + e.getValue());  
}
```

✅ 3. Using Iterator

```
Iterator<Map.Entry<Integer, String>> it = map.entrySet().iterator();
```

🔥 Important Point

“Map.Entry is an inner interface used to represent key-value pair.”

🚀 Final Interview Closing Answer (your style)

“HashMap is a non-synchronized Map implementation that stores key-value pairs using hashing, allows one null key, and uses load factor to manage resizing.”

? HashMap Internal Working (Step-by-Step)

📖 Importance: ★★★★★ (Top Interview Question)

✅ Interview-ready answer (your style, clean & complete)

“HashMap stores data using hashing. When we insert a key-value pair, first the key’s hashCode() is generated. This hash value is converted into a bucket index, and that index is the position where the entry will be stored in the internal array.

If the bucket is empty, the entry is stored directly. If the bucket is not empty, then equals() is used to compare keys. If the same key is found, the value is updated; otherwise, the new entry is added in the same bucket using a linked list or tree (in newer versions)."

? What is TreeMap

 Importance: ★★★★★ (Very High)

✅ Clean & Correct Definition (your style)

"TreeMap is a class that implements the NavigableMap interface and indirectly implements SortedMap and Map. Its underlying data structure is a Red-Black Tree. It stores data in key-value pairs and maintains sorting order based on keys. It does not maintain insertion order, does not allow null keys (but allows null values), and is not synchronized, so it is not thread-safe."

✗ Small Corrections

- ✗ "cannot store null"
👉 ✅ No null key, but null values allowed
 - ✓ Sorting is based on:
 - Comparable (default)
 - Comparator (custom)
-

? Properties of Red-Black Tree

 Importance: ★★★★★

👉 Interview answer (your style):

"Red-Black Tree is a self-balancing binary search tree that maintains balance to ensure efficient operations."

🔥 Key Properties:

- Each node is either **Red or Black**
- Root node is always **Black**
- A red node cannot have a red parent or red child. This rule helps maintain balance in the tree.
- Every path has same number of **Black nodes**

- Keeps tree **balanced** ($O(\log n)$ operations)
-

? Constructors of TreeMap

 Importance: ★★★★★

```
TreeMap()                // natural sorting
TreeMap(Comparator comp) // custom sorting
TreeMap(Map m)            // from another map
TreeMap(SortedMap m)      // from sorted map
```

? Important Methods

 Importance: ★★★★★

```
put(K key, V value)      // add/update
get(Object key)          // get value
remove(Object key)       // remove entry
containsKey(Object key)  // check key
containsValue(Object v)  // check value
size()                  // number of entries
firstKey()               // smallest key
lastKey()                // largest key
```

🔥 Navigable Methods (Very Important)

```
higherKey(K key)        // next greater key
lowerKey(K key)          // next smaller key
ceilingKey(K key)        // >= key
floorKey(K key)          // <= key
pollFirstEntry()         // remove first
pollLastEntry()          // remove last
```

🎯 Final Interview Closing Answer (your style)

“TreeMap is a sorted Map implementation based on Red-Black Tree that stores key-value pairs in sorted order, does not allow null keys, and provides navigation methods for efficient data access.”

? What is Hashtable

 Importance: ★★★★★ (High)

 Interview-ready answer (your style, clean):

“Hashtable is a class that implements the Map interface. Its underlying data structure is a hash table. It stores data in key-value pairs, does not allow null key or null value, does not maintain insertion or sorting order, and is synchronized, so it is thread-safe.”

? Hash Collision

 Importance: ★★★★★

 Answer (your style):

“Hash collision occurs when two different keys generate the same bucket index. In that case, both entries are stored in the same bucket using chaining (linked list).”

? Retrieval Technique

 Importance: ★★★★★

 Answer (your style):

“Data in Hashtable can be retrieved using `keySet()`, `entrySet()`, or `Enumeration`. We can traverse from first bucket to last bucket internally and access key-value pairs.”

 Ways:

```
// Using keySet
for (Integer key : map.keySet()) {
    System.out.println(map.get(key));
}

// Using entrySet (best)
for (Map.Entry<Integer, String> e : map.entrySet()) {
    System.out.println(e.getKey() + " " + e.getValue());
}

// Using Enumeration (legacy)
Enumeration<Integer> keys = ht.keys();
```

? Constructors

 Importance: ★★☆☆

```
Hashtable() // default capacity (11) and load factor (0.75)
Hashtable(int initialCapacity) // custom capacity
Hashtable(int initialCapacity, float loadFactor)
Hashtable(Map m) // create from another map
```

? Important Methods

 Importance: ★★★★★

```
put(K key, V value) // add or update entry
get(Object key) // retrieve value
remove(Object key) // remove entry
containsKey(Object key) // check key exists
containsValue(Object v) // check value exists
size() // number of entries
isEmpty() // check empty
clear() // remove all entries
```

```
keys() // returns Enumeration of keys
elements() // returns Enumeration of values
```

? Difference between HashMap and Hashtable

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“HashMap and Hashtable both store data in key-value pairs using hashing, but HashMap is not synchronized and faster, while Hashtable is synchronized and thread-safe but slower. Also, HashMap allows one null key and multiple null values, whereas Hashtable does not allow any null key or value.”

🔥 Quick Difference:

Feature	HashMap	Hashtable
Thread Safety	✗ Not synchronized	☑ Synchronized
Performance	Fast	Slower
Null Key	☑ One allowed	✗ Not allowed
Null Value	☑ Allowed	✗ Not allowed
Legacy	✗ Modern	☑ Legacy

? Difference between HashSet and TreeSet

 Importance: ★★★★★ (Very High)

 Interview-ready answer (your style, clean):

“HashSet and TreeSet are both Set implementations, but HashSet does not maintain any order and uses hashing, while TreeSet maintains sorted order and uses a Red-Black Tree.”

🔥 Quick Difference:

Feature	HashSet	TreeSet
Order	✗ No order	☑ Sorted order
Underlying DS	Hash table	Red-Black Tree
Performance	Faster ($O(1)$)	Slower ($O(\log n)$)
Null	☑ One allowed	✗ Not allowed
Duplicate	✗ Not allowed	✗ Not allowed

? Difference between List, Set, and Map

 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“List, Set, and Map are part of the Collection Framework. List stores elements with duplicates and maintains insertion order, Set stores unique elements without duplicates, and Map stores data in key-value pairs where keys are unique.”

🔥 Quick Difference:

Feature	List	Set	Map
Data	Single values	Single values	Key-value pairs
Duplicate	✅ Allowed	❌ Not allowed	❌ Keys not allowed (values allowed)
Order	✅ Maintains order	❌ No guarantee (except LinkedHashSet)	Depends on implementation
Access	By index	By element	By key
Index	✅ Yes	❌ No	❌ No

? Difference between HashMap, LinkedHashMap, and TreeMap

📊 Importance: ★★★★★ (Very High)

👉 Interview-ready answer (your style, clean):

“HashMap, LinkedHashMap, and TreeMap are Map implementations. HashMap does not maintain any order, LinkedHashMap maintains insertion order, and TreeMap maintains sorted order based on keys.”

🔥 Quick Difference:

Feature	HashMap	LinkedHashMap	TreeMap
Order	❌ No order	✅ Insertion order	✅ Sorted order
Underlying DS	Hash table	Hash table + linked list	Red-Black Tree

Feature	HashMap	LinkedHashMap	TreeMap
Performance	Fastest (O(1))	Slightly slower	Slowest (O(log n))
Null Key	✅ One allowed	✅ One allowed	❌ Not allowed
Thread Safety	❌ Not synchronized	❌ Not synchronized	❌ Not synchronized

? What is ConcurrentHashMap

📖 Importance: ★★★★★ (High)

👉 Interview-ready answer (your style, clean):

“ConcurrentHashMap is a class in Java used to store key-value pairs in a thread-safe way. It allows multiple threads to read and write data simultaneously without locking the entire map, so it provides better performance than Hashtable.”

🔥 Key Properties:

- ✅ Thread-safe (concurrent)
- ⚡ Better performance than Hashtable
- ❌ Does not allow null key or null value
- 🔄 Allows multiple threads to work simultaneously
- Uses **segment-level / bucket-level locking** (not full lock)

🔥 Simple Understanding:

- **Hashtable** → locks whole map (slow)
- **ConcurrentHashMap** → locks only part (fast)

Java 8 features

? What is Lambda Expression

 Importance: ★ ★ ★ ★ ★

 Interview-ready answer (your style):

“Lambda expression is a feature in Java 8 that allows us to write anonymous functions in a shorter and cleaner way. It is mainly used to implement functional interfaces.”

🔥 Why we use it

- Reduce boilerplate code
 - Replace anonymous class
 - Improve readability
-

🔥 Syntax

(parameters) -> expression

🔍 Example

✗ Before Java 8

```
</> Java

Runnable r = new Runnable() {
    public void run() {
        System.out.println("Hello");
    }
};
```

✓ Java 8

```
</> Java

Runnable r = () -> System.out.println("Hello");
```

? What is Functional Interface

 Importance: ★ ★ ★ ★ ★ (Very High)

✅ Interview-ready answer (your style, clean & complete)

"Functional Interface is an interface that contains exactly one abstract method. It is used as the target type for lambda expressions. It can have multiple default and static methods, but only one abstract method."

🔥 Key Points

- ✅ Only **one abstract method**
 - ✅ Can have **multiple default & static methods**
 - ✅ Used with **lambda expressions**
 - ✅ Annotated with `@FunctionalInterface` (optional but recommended)
-

🔥 Syntax Example

```
@FunctionalInterface
interface MyInterface {
    void show(); // only one abstract method
}
```

🔥 Lambda Usage

```
MyInterface obj = () -> System.out.println("Hello");
obj.show();
```

🔥 Predefined Functional Interfaces (Very Important)

🔗 From `java.util.function` package:

- `Function<T, R>` → takes input, returns output
- `Predicate<T>` → returns boolean
- `Consumer<T>` → takes input, no return
- `Supplier<T>` → no input, returns value

Example

```
Predicate<Integer> p = x -> x > 10;  
System.out.println(p.test(20)); // true
```

Why we use it

- To support **Lambda Expressions**
- To enable **functional programming in Java**
- To write **clean and concise code**

1. Function<T, R>

 Importance: ★★★★★

Definition

"Function takes one input and returns one output."

Method

```
R apply(T t)
```

Example

 Java

```
Function<Integer, Integer> f = x -> x * 2;  
System.out.println(f.apply(5)); // 10
```

Use Case

- Data transformation
 - Mapping values (used in Stream API)
-

Interview Line

"Function is used when we need to take input and return output."

2. Predicate<T>

 Importance: ★★★★★

Definition

"Predicate takes input and returns boolean (true/false)."

Method

```
boolean test(T t)
```

Example

 Java

```
Predicate<Integer> p = x -> x > 10;  
System.out.println(p.test(20)); // true
```

Use Case

- Filtering data (very important in streams)
-

Interview Line

"Predicate is used for condition checking and returns boolean."

3. Consumer<T>

 Importance: ★★★★★

 Definition

"Consumer takes input but does not return anything."

 Method

```
void accept(T t)
```

Example

 Java

```
Consumer<String> c = s -> System.out.println(s);  
c.accept("Hello");
```

 Use Case

- Printing/logging
 - Performing operations
-

 Interview Line

"Consumer is used when we only need to process data without returning anything."

4. Supplier<T>

 Importance: ★★★★★

 Definition

"Supplier does not take input but returns a value."

Method

```
</> Java
```

```
T get()
```

Example

```
</> Java
```

```
Supplier<Double> s = () -> Math.random();  
System.out.println(s.get());
```

Use Case

- Generate values
 - Lazy initialization
-

Interview Line

"Supplier is used when we need to return data without taking any input."

Final Summary (VERY IMPORTANT)

Interface Input Output

Function ☒ Yes ☒ Yes

Predicate ☒ Yes ☒ Boolean

Consumer ☒ Yes ☒ No

Interface Input Output

Supplier ☒ No ☐ Yes

? Stream API in Java 8

 Importance: ★★★★★ (Very High)

"Stream API is used to process collections in a functional and efficient way. It allows us to perform operations like filter, map, sort, and reduce on data without changing the original collection."

🔥 Why we use Stream API

- to process data in a clean way
 - to write less code
 - to support functional style programming
 - to work on collections easily
-

🔥 Basic idea

"A stream is not a data structure. It is a pipeline of operations on data."

🔥 Simple example

```
List<Integer> list = Arrays.asList(10, 20, 30, 40);  
  
list.stream()  
    .filter(x -> x > 20)  
    .forEach(System.out::println);
```

👉 Here:

- stream() creates stream
- filter() selects data

- `forEach()` prints data
-

🔥 Important Stream methods

- `filter()` → select elements
- `map()` → transform elements
- `sorted()` → sort elements
- `forEach()` → process each element
- `collect()` → collect result into List/Set/etc.
- `reduce()` → combine elements into one result

Stream methods in brief

`filter()`

"Used to select elements based on a condition."

Example: keep only numbers greater than 10.

`map()`

"Used to transform each element into another form."

Example: convert numbers to their squares.

`sorted()`

"Used to arrange elements in ascending or custom order."

Example: sort names alphabetically.

`forEach()`

"Used to process or print each element one by one."

Example: print all elements.

`collect()`

"Used to collect stream results into a List, Set, or another collection."

Example: convert stream back to list.

`reduce()`

"Used to combine all elements into one result."

Example: sum of all numbers.

? forEach() method in Java 8

 Importance: ★★★★★ (Very High)

✅ Interview-ready answer (your style)

"forEach() is a method used to iterate over elements of a collection or stream and perform an action on each element."

🔥 Syntax

```
collection.forEach(element -> action);
```

🔍 Example

```
List<Integer> list = Arrays.asList(10, 20, 30);  
  
list.forEach(x -> System.out.println(x));
```

👉 Output:

```
10  
20  
30
```

💧 Using Method Reference (clean way)

```
list.forEach(System.out::println);
```

🔥 Where it is used

- Collections (List, Set)
 - Streams
-

🔥 Key Points

- ☒ Used for iteration
- ☒ Uses lambda expression
- ☒ Does not return anything
- ☒ Cannot modify structure (like add/remove safely)

? map() vs flatMap()

📊 Importance: ★★★★★ (Very High)

☒ map()

👉 Interview answer (your style):

"map() is used to transform each element into another form, and it returns one output for each input."

🔍 Example

```
List<Integer> list = Arrays.asList(1, 2, 3);

list.stream()
    .map(x -> x * 2)
    .forEach(System.out::println);
```

👉 Output:

2 4 6

☒ flatMap()

👉 Interview answer (your style):

"flatMap() is used to flatten nested data structures and convert them into a single stream."

🔍 Example

```
List<List<Integer>> list = Arrays.asList(
    Arrays.asList(1, 2),
    Arrays.asList(3, 4)
);

list.stream()
    .flatMap(x -> x.stream())
    .forEach(System.out::println);
```

👉 Output:

1 2 3 4

🔥 Key Difference

Feature	map()	flatMap()
Output	One-to-one	One-to-many (flatten)
Use	Simple transformation	Flatten nested data

? Method Reference in Java 8

📊 Importance: ★★★★★ (High)

✅ Interview-ready answer (your style)

"Method reference is a shorthand way of writing lambda expressions by directly referring to an existing method."

🔥 Why we use it

- Reduce code
- Improve readability
- Replace simple lambda expressions

🔥 Syntax

```
ClassName::methodName
```

💧 Types of Method Reference

◇ 1. Static Method Reference

```
ClassName::staticMethod
```

👉 Example:

```
Integer::parseInt
```

◇ 2. Instance Method Reference (of object)

```
object::methodName
```

👉 Example:

```
System.out::println
```

◇ 3. Instance Method Reference (of class)


```
ClassName::methodName
```

👉 Example:

```
String::toUpperCase
```

◊ 4. Constructor Reference

```
ClassName::new
```

👉 Example:

```
ArrayList::new
```

🔍 Example

```
List<String> list = Arrays.asList("a", "b", "c");  
list.forEach(System.out::println);
```

👉 Instead of:

```
list.forEach(x -> System.out.println(x));
```

? What is Optional in Java 8

📊 Importance: ★★★★★ (Very High)

✅ Interview-ready answer (your style, clean & complete)

"Optional is a container class in Java 8 used to handle null values safely. It helps avoid NullPointerException by providing methods to check and handle absence of values."

🔥 Why we use Optional

- Avoid `NullPointerException` (NPE)
 - Write clean and safe code
 - Force developer to handle null cases properly
-

🔥 Creating Optional

```
Optional<String> op1 = Optional.of("Hello");           // value must not be null
Optional<String> op2 = Optional.ofNullable(null);       // can be null
Optional<String> op3 = Optional.empty();               // empty optional
```

🔥 Important Methods

◇ Check value

```
op.isPresent()    // true/false
```

◇ Get value

```
op.get()    // risky (throws exception if empty)
```

◇ Safe access (VERY IMPORTANT)

```
op.orElse("Default")
op.orElseGet(() -> "Default")
op.orElseThrow(() -> new RuntimeException())
```

◇ Perform action

```
op.ifPresent(x -> System.out.println(x));
```

◇ Transform value

```
op.map(String::toUpperCase)
```

Example

```
Optional<String> name = Optional.ofNullable(null);  
  
String result = name.orElse("Default Name");  
System.out.println(result);
```

 Output:

Default Name

Key Points

-  Avoid using `get()` directly
-  Use `orElse()` / `ifPresent()`
-  Used in **Spring Boot APIs** (very common)

Common Mistakes

- Using `Optional` for **fields** (not recommended)
- Using `get()` without checking

Default Methods in Interface

 Importance: ★ ★ ★ ★

 Interview Answer (your style)

"Default methods are methods defined inside an interface with a body using the `default` keyword. They allow adding new methods to interfaces without breaking existing implementations."

🔥 Why introduced

- To maintain **backward compatibility**
 - So old classes don't break when new methods are added
-

🔍 Example

```
interface MyInterface {  
    default void show() {  
        System.out.println("Default method");  
    }  
}
```

🔥 Key Points

- ☒ Has method body
 - ☒ Can be overridden
 - ☒ Used for extending interfaces safely
-

🎯 Final Line

"Default methods allow adding functionality in interfaces without breaking existing code."

? Static Methods in Interface

📊 Importance: ★ ★ ★

☒ Interview Answer

"Static methods in interface belong to the interface itself and can be called using interface name."

🔍 Example

```
interface MyInterface {  
    static void display() {  
        System.out.println("Static method");  
    }  
}
```

👉 Call:

```
MyInterface.display();
```

🔥 Key Points

- ❌ Cannot be overridden
- ✅ Called using interface name
- ✅ Used for utility methods

🎯 Final Line

"Static methods in interface are utility methods that belong to interface and cannot be overridden."

? Date & Time API (java.time)

📊 Importance: ★★★★★

✅ Interview Answer (your style)

"Java 8 introduced a new Date and Time API in java.time package to handle date and time in a better, immutable, and thread-safe way."

🔥 Why introduced

- Old Date API had issues
- Not thread-safe

- Difficult to use

🔥 Important Classes

- ◆ **LocalDate** (Date only)

```
LocalDate date = LocalDate.now();
```

- ◆ **LocalTime** (Time only)

```
LocalTime time = LocalTime.now();
```

- ◆ **LocalDateTime** (Date + Time)

```
LocalDateTime dt = LocalDateTime.now();
```

- ◆ **DateTimeFormatter** (Formatting)

```
DateTimeFormatter f = DateTimeFormatter.ofPattern("dd-MM-yyyy");  
String formatted = date.format(f);
```

🔍 Example

```
LocalDate date = LocalDate.now();  
System.out.println(date);
```

🔥 Key Points

- ☒ Immutable (cannot change)
- ☒ Thread-safe
- ☒ Easy formatting